

CS336 作业 5（对齐）：推理 RL

版本26.0.0

CS336工作人员

2026 年春季

1 作业概述

在本次作业中，您将获得一些训练语言模型来推理和解决下游任务的实践经验。

您将实施什么

1. 零镜头、少镜头、链式提示。2. 组相对策略优化（GRPO），一种强化学习算法，用于在给定外部奖励的情况下提高模型性能。3. 策略梯度估计变体，探索方差减少和重要性权重裁剪策略。

你将运行什么

1. 测量 GSM8K 上 OLMo-2-0425-1B 的提示性能。2. 在 OLMo-2-0425-1B 上运行按策略 GRPO 以提高 GSM8K 的性能。3. 运行 RL 变体，包括 RFT、Dr. GRPO 和 MaxRL，以探索 RL 中的算法选择。4. 运行 off-policy GRPO，以加快训练速度并探索各种裁剪策略。

代码是什么样的

所有作业代码以及这篇文章都可以在 GitHub 上找到：github.com/stanford-cs336/assignment5-alignment。请 `git clone` 存储库。如果有任何更新，我们会通知您，您可以 `git pull` 获取最新信息。

1. `cs336_alignment/`：您将在此处编写作业 5 的代码。请注意，这里没有任何代码（除了下面描述的起始代码），因此您应该能够从头开始做任何您想做的事情。2. `cs336_alignment/vllm_utils.py`：此文件包含运行 vLLM 服务器的代码，该服务器生成代码并可以同步权重。3. `cs336_alignment/drgrpo_grader.py`：此文件包含用于对数学问题的模型输出进行评分的代码。4. `cs336_alignment/prompts/`：为了您的方便，我们提供了带有提示的文本文件。5. `tests/*.py`：此目录包含必需的 GRPO 测试和可选的对齐/安全补充测试。

`tests/test_grpo.py` 中所需的测试调用 `tests/adapters.py` 中定义的钩子。您将实现适配器以将代码连接到测试。编写更多测试和/或修改测试代码有助于调试代码，但您的实现预计会通过原始提供的测试套件。

1. `README.md`：此文件包含有关设置环境的一些基本说明。

如何提交

您将向 Gradescope 提交以下文件：

- `writeup.pdf`: 回答所有书面问题。请排版您的回复。
- `code.zip`: 包含您编写的所有代码。

运行 `test_and_make_submission.sh` 中的脚本以创建 `code.zip` 文件。

2 简介

2.1 背景

在本课程的前四项作业中，我们学习了如何预训练基本模型。我们现在准备学习后期训练：一旦我们有了基本模型，我们如何将其变成可以解决下游任务的有用工具？

后训练的组成部分之一是 *alignment*。预训练由于其数据混合和训练目标，具有为模型注入广泛的知识的行为的效果。但是，当我们要求模型解决任务时，我们需要一个特定的行为，即一个有帮助且无害的助手的行为。将广泛的基础模型转变为重点聊天模型的过程称为“指令调整”或“对齐”，我们将在可选的作业 5 补充中介绍这些技术。

后训练的另一个组成部分是 *reinforcement learning*。在预训练中，因为我们的目标是为模型注入广泛的知识库，所以我们使用广泛的数据集（网络上的文本）。在训练后强化学习中，我们的目标缩小了：我们希望模型在特定任务上获得高精度，例如解决数学问题。这个较窄的目标至少在两个方面与预训练不同：（a）我们不再拥有那么多的数据，（b）我们的训练目标从“覆盖范围”（拥有广泛的知识库）之一变为“精度”（产生准确的响应）之一。由于这些差异，我们需要一种新技术，即强化学习（RL）。

在强化学习中，我们得到一个任务数据集和一个评分函数，用于判断响应是否正确解决了问题。例如，在编码设置中，我们可能会遇到一个编码问题（例如“编写一个反转列表的 Python 函数”），以及由一组测试用例组成的评分函数（例如 `assert f([0, 1, 2]) == [2, 1, 0]`）。或者在数学环境中，我们可能会遇到一个数学推理应用题（例如，“如果四年前汤姆的年龄是约翰的一半，而约翰现在二十岁，那么汤姆现在多大了？”），以及一个解析最终答案并检查其是否正确的评分函数（12）。值得注意的是，与预训练不同，我们没有得到模仿反应的数据集：在编码示例中，我们没有得到解决任务的正确 Python 程序，在数学推理示例中，我们没有得到产生答案的正确推理链。相反，在强化学习中，我们直接采用模型精度的梯度步骤作为目标函数。在较高层面上，强化学习将涉及对模型响应进行采样，使用评分函数对它们进行评分，然后对正确的响应进行加权。

在本次作业中，我们将从数学和经验上了解强化学习。事实证明强化学习很困难，因为它缓慢且不稳定。对 RL 进行科学研究也很困难，因为 RL 运行在随机种子之间具有很高的方差，而且实现中看似很小的细节却很重要。本作业将介绍 LLM RL 并探讨其一些挑战。

2.2 模型和数据集

对于此任务，我们将使用 OLMo-2-0425-1B，这是一个在 OLMo-mix-1124 上进行预训练并在 Dolmino-mix-1124 上进行中期训练的基本模型，总共有 4 万亿个令牌。OLMo-mix-1124 主要由 DCLM-Baseline 组成 [J. Li et al., 2024]，这是我们在讲座中介绍的数据集，而 Dolmino-mix-1124 是一个更集中的数据混合物，包含大约 50% DCLM 和 50% 指令跟踪、数学、代码、STEM 论文和 wiki 数据。其他 OLMo-2-0425-1B 详细信息可以在 OLMo 2 中找到

技术报告[T. OLMo 等人, 2024]。此时, 在课堂上, 您应该具备理解他们的每个设计决策所需的所有背景。

作为我们的下游任务, 我们将使用 GSM8K 数据集 [K. Cobbe et al., 2021], 可在 data/gsm8k/train.jsonl 和 data/gsm8k/test.jsonl 的作业存储库中获取, 或在此处在线获取。该数据集包含相对简单的小学数学推理应用题集合。示例如下:

```
{
  "question": "Natalia sold clips to 48 of her friends in April, and then she sold half as many clips in May. How many clips did Natalia sell altogether in April and May?",
  "answer": "Natalia sold 48/2 = <<48/2=24>>24 clips in May.\nNatalia sold 48+24 = <<48+24=72>>72 clips altogether in April and May.\n#### 72"
}
```

在强化学习过程中, 我们的模型将学习为此类问题生成推理链, 从而提高其解决数学问题的能力。

关于模型和数据集选择的说明: 由于资源限制, 我们只选择了一个小规模模型-数据集组合作为我们的 RL 测试平台。特别是, 我们选择了一个非常强大的小型模型, 因为它接受了大量标记 (参数数量的 4000 倍!) 的训练, 使我们能够在现实数据集上看到小规模模型的合理 RL 结果。不幸的是, 我们在本课程中训练的模型太弱, 无法解决数学问题。如果您在完成此作业后感到好奇, 也欢迎您使用您在本课程中训练的模型并在更简单的任务上运行强化学习。

由于 RL 训练动态非常依赖于模型和数据集, 因此您应该记住, 我们在此作业中看到的结果可能不会转移到其他模型-数据集组合。

2.3 符号

该作业将包含一些数学内容, 因此在下表中我们提供了一些稍后将使用的符号。语言建模和强化学习通常对同一对象使用不同的术语, 因此该表包含这两个术语。如果您忘记了某个符号的含义, 请随时返回此表。

Notation	LM terminology	RL terminology	Meaning
ρ	prompt distribution or dataset	initial state distribution	Distribution over prompts/problems.
x	prompt; question	initial state	A prompt/problem sampled from ρ .
y	response; completion; generation; sample	rollout; trajectory; sampled action sequence	A sampled answer to prompt x .
y_t	token	action	The t -th generated token in y .
$y_{<t}$	prefix		All generated tokens before position t , or $y_1, \dots, y_{t-1}$.
π_θ	model	policy	A model with parameters θ , which assigns probabilities $\pi_\theta(y x)$ to responses y given prompts x .

$\pi_\theta(y_t \mid x, y_{<t})$	next-token distribution	policy at timestep t	Conditional probability of token y_t given the prompt and previous generated tokens.
$r(y \mid x)$		reward	Scalar score denoting the correctness of a sampled response (0 or 1 in this assignment).
B	number of prompts per batch		Number of prompts per inference batch.
G	generations per prompt	group size	Number of responses sampled for each prompt.
$\text{len}(y)$ or L	response length	horizon	Number of generated tokens in a response.
$A^{(i,j)}$		advantage	Weight assigned to response j for prompt i after baseline and normalization.

3 提示

将预训练基础模型应用于下游任务的第一步是提示它。基础模型针对广泛的行为进行了预训练，而提示是一种将其行为塑造为任务解决模式的轻量级方法。在稍后的作业中，我们还将看到即时选择塑造了强化学习的动态和探索。

提示模型最基本的方法是提供问题，并从其下一个令牌分布中采样以产生答案；我们将这个策略称为 `question_only`。我们将此策略与 `r1_zero` 提示进行比较，其中包括模型进行思想链推理的问题和说明 [DeepSeek-AI et al., 2025]。



3.1 使用vLLM进行推理

为了从模型中生成响应，我们需要一个推理引擎。实现推理引擎超出了本次作业的范围，因此我们将使用 `vLLM` 推理引擎 [W. Kwon et al., 2023]，它实现了各种优化，包括快速 CUDA 内核、用于高效注意力 KV 缓存的 PagedAttention 等。`cs336_alignment/vllm_utils.py` 中提供了启动 `vLLM` 服务器并产生生成代的代码，该代码具有以下接口：

```
@dataclass
class VLLMCompletion:
    text: str
    token_ids: list[int]
    finish_reason: str | None

@dataclass
class VLLMServer:
    model_id: str
    gpu: int = 0
    seed: int = 0
    gpu_memory_utilization: float = 0.9
```

```
def start(self) -> None: ...

def generate_completions(
    self,
    prompts: list[str],
    sampling_params: dict,
    batch_size: int | None = None,
) -> list[VLLMCompletion]: ...
```

3.2 零样本、少样本和思维链提示

除非另有说明，对于 GSM8K 的实验，我们将使用 DeepSeek R1-Zero 模型中的以下提示 [DeepSeek-AI et al., 2025]。我们将其称为 `r1_zero` 提示符：

```
A conversation between User and Assistant. The User asks a question, and the Assistant solves
it. The Assistant first thinks about the reasoning process in the mind and then provides the
User with the answer. The reasoning process is enclosed within <think> </think> and the
answer is enclosed within <answer> </answer> tags, respectively, i.e., <think> reasoning
process here </think> <answer> answer here </answer>.
User: {question}
Assistant: <think>
```

`r1_zero` 提示符位于文本文件 `cs336_alignment/prompts/r1_zero.prompt` 中。

在提示中，`question` 指的是我们插入的一些问题（例如，`Natalia sold clips to 48 of her friends in April, and then she sold half as many clips in May. How many clips did Natalia sell altogether in April and May?`）。我们期望模型扮演助手的角色，并开始生成思维过程（因为我们已经包含了一个开始的 `<think>` 标签），用 `</think>` 结束思维过程，然后在答案标签内生成最终的符号答案，例如 `<answer> 4x + 10 </answer>`。让模型生成像 `<answer> </answer>` 这样的标签的目的是为了让我们可以轻松解析模型的输出并将其与真实答案进行比较，这样当我们看到结束答案标签 `</answer>` 时我们就可以停止生成响应。

另一种提示方法称为“少样本提示”，是在用实际问题提示模型之前预先添加一些问答对。`r1_zero` 提示的几个版本如下所示：

```
A conversation between User and Assistant. The User asks a question, and the Assistant solves
it. The Assistant first thinks about the reasoning process in the mind and then provides the
User with the answer. The reasoning process is enclosed within <think> </think> and the
answer is enclosed within <answer> </answer> tags, respectively, i.e., <think> reasoning
process here </think> <answer> answer here </answer>.
User: {question-1}
Assistant: <think> {reasoning-1} </think> <answer> {answer-1} </answer>
User: {question-2}
Assistant: <think> {reasoning-2} </think> <answer> {answer-2} </answer>
User: {question-3}
Assistant: <think> {reasoning-3} </think> <answer> {answer-3} </answer>
User: {question}
Assistant: <think>
```

少量提示通过提供一些要解决的任务示例来提高模型的性能。开放基础模型报告的基准数据通常是小样本：例如，OLMo-2-0425-1B 模型卡上的 GSM8K 报告的结果使用 8 样本提示。我们在 `cs336_alignment/prompts/r1_zero_three_shot_gsm8k.prompt` 中包含了 `r1_zero` 提示符的 3 个版本，示例取自 OLMES 存储库。

最后，作为基准，我们将包含 `question_only` 提示符，可在 `cs336_alignment/prompts/question_only.prompt` 中找到：

```
{question} Please put your final answer within \boxed{{}}.
```

请注意，虽然提示名为 `question_only`，但我们要求将最终答案装箱，以便评分者更容易从响应中解析答案，如下一节所述。

3.3 评分功能

一旦模型生成响应，我们需要检查它是否正确。我们的数学问题包括真实答案（例如 0.5），但模型可以通过多种方式提供正确答案：例如，它可以回答 `<answer> 1/2 </answer>` 或 `The answer is 0.5`。因此，为了正确地对模型输出进行评分，我们需要一个答案解析函数，该函数将模型的输出和已知的事实真相作为输入，并返回一个布尔值，指示模型的输出是否正确。

对于我们的实验，我们将使用最近推理工作中使用的快速且相当准确的答案解析器

RL [Z. 刘等人, 2025]。对于 `r1_zero` 提示，此奖励函数在 `cs336_alignment.drgro_grader.r1_zero_reward_fn` 处实现。`question_only` 提示不会要求模型使用 `<think>` 和 `<answer>` 标签，因此应该使用

`cs336_alignment.drgro_grader.question_only_reward_fn` 来自同一个评分者文件。请注意，`question_only` 解析器在 `\boxed{{}}` 中寻找最终答案。这些奖励函数返回总奖励，以及单独的格式和答案奖励，指示响应是否遵循预期格式以及解析的答案是否正确。有些作品给予部分学分（非零总奖励）

对于格式正确但不正确的响应。在我们的实验中，我们不会给予部分计分，因此总奖励只是答案奖励，并且我们只会使用格式奖励进行记录。

请注意，这些奖励函数期望 `ground_truth` 参数仅包含答案。要从包含 `{rationale} #### {answer}` 的 GSM8K 数据集中的真实响应中提取答案，您应该在 `####` 上进行拆分并去除空格。

3.4 实验

我们现在准备评估基本模型的提示性能。

生成超参数。生成响应时，我们将以温度 1.0、top-p 1.0、最大生成长度 512 进行采样。`r1_zero` 提示要求模型以字符串 `</ answer>` 结束其答案，因此在使用这些提示时，我们可以指示 vLLM 在模型输出此字符串时停止：

```
# Based on Dr. GRP0: stop when the model completes its answer
# https://github.com/sail-sg/understand-r1-zero/blob/
# c18804602b85da9e88b4aeeb6c43e2f08c594fbc/train_zero_math.py#L167
sampling_params['stop'] = ["</answer>"]
sampling_params['include_stop_str_in_output'] = True
```

仅将此停止字符串用于 `r1_zero` 和 `r1_zero_three_shot` 提示符，而不用于 `question_only` 提示符。

问题 (prompting_baselines): 在 GSM8K 上运行 OLMo-2-0425-1B (5 分)

(a) 编写一个脚本，使用零样本 question_only、零样本 r1_zero 和少样本 r1_zero_three_shot 提示来评估 OLMo-2-0425-1B 在 GSM8K 上的性能。

然后，运行脚本并观察输出。对于每个提示，有多少模型代属于以下类别：(1) 格式和正确性奖励 1 均正确，(2) 格式奖励 1 和正确性奖励 0，(3) 格式奖励 0 和正确性奖励 0？观察至少 10 个类别 2 的示例，有多少模型输出实际上是正确的，但只是未正确解析？那么第 3 类呢？

可交付成果：几句话评论、评估指标以及一些提示和响应的示例。

(b) 观察模型输出，描述模型在每个提示下的行为。例如，如果我们希望模型回答问题，那么仅仅提供问题就足够了，还是模型除了回答问题之外还表现出其他行为？零样本 r1_zero 和少样本 r1_zero_three_shot 提示如何塑造模型的行为？

可交付成果：几句带有支持示例的评论。

4 集团相关政策优化

现在我们已经通过提示测量了模型的性能，下一步将是通过训练来提高该性能。具体来说，我们希望优化模型的准确性，或者等效地，优化其预期奖励：

$$J_{\theta} = E_{x \sim \rho} E_{y \sim \pi_{\theta}(y | x)} [r(y | x)], \quad (1)$$

其中 ρ 是我们对提示/问题 x 的任务分布， π_{θ} 是我们的模型， y 是对 x 的采样响应/解决方案， $r(y | x)$ 表示 y 是否是 x 的正确答案（我们也将 r 称为我们的“奖励函数”）。请注意，这个目标与我们迄今为止在课堂上研究的预训练目标有很大不同：与我们从数据集 $\mathcal{D}$ 中抽取样本的交叉熵损失 $E_{x \sim \mathcal{D}} [-\log \pi_{\theta}(x)]$ 不同，我们的准确性目标涉及对模型本身样本的期望。因此，我们需要新的工具来优化这一目标：特别是，我们需要使用强化学习（RL）。在较高的层面上，强化学习将涉及对模型的响应进行采样，并使用评分函数对它们进行评分，

并强化正确的内容。强化学习通常很缓慢且具有挑战性，因此这项任务的大部分内容将涉及探索工具以使其更快、更稳定。

强化学习的另一方面是，因为它涉及从模型中采样，所以它的动态取决于采样过程中使用的提示。特别是，语言模型研究中的一个令人兴奋的发现是，当使用思想链推理提示时，在强大的基础模型上执行强化学习可以导致其推理能力的显著提高[OpenAI et al., 2024; DeepSeek-AI 等人, 2025]。因此，在本次作业中，除了探索强化学习的核心算法思想外，我们还将探索其与思想链提示等语言建模技术的交互。

4.1 导出同策略GRPO

我们将首先探索 RL 背后的数学原理，构建组相对策略优化 (GRPO) 算法，这是训练语言模型时常用的 RL 算法[Z.邵等人, 2024]。

我们的演示紧密基于一些很棒的资源，这些资源更深入地介绍了这些概念：OpenAI 的 *Spinning Up in Deep RL* [J. Achiam, 2018] 和 Nathan Lambert 的《人类反馈强化学习》(RLHF) 书 [N.兰伯特, 2024]。

4.1.1 作为策略的语言模型

传统上，在强化学习中，我们认为 *policy* 接收 *state* s_t 并输出 *action* a_t 。然后，此操作会产生一些 *reward* $r_t = r(s_t, a_t)$ ，以及从某个下一个状态分布 $s_{t+1} \sim \text{next_state}(s_t, a_t)$ 得出的后续状态。强化学习涉及优化策略以获得高奖励。

具有参数 θ 的因果语言模型 (LM) π_θ 定义了给定当前文本前缀 $y_{<t} = (y_1, \dots, y_{t-1})$ 的下一个标记 y_t 的概率分布。因此，在 RL 的上下文中，我们可以将下一个标记 y_t 视为我们的 *action* a_t ，将当前文本前缀 $y_{<t}$ 视为 *state* s_t 。因此，LM 是一个 *categorical stochastic policy*

$$a_t \sim \pi_\theta(\cdot | s_t), \pi_\theta(a_t | s_t) = [\text{softmax}(f_\theta(s_t))]_{a_t}. \quad (2)$$

当使用策略梯度优化策略时，我们需要两个原始操作：

1. *Sampling from the policy*: 从上面的分类分布中得出一个动作 a_t ;
2. *Scoring the log-likelihood of an action*: 评估 $\log \pi_\theta(a_t | s_t)$ 。

通常，当使用 LLM 进行 RL 时， s_t 是迄今为止产生的部分完成/解决方案，每个 a_t 是解决方案的下一个标记；当发出文本结束标记时，该剧集结束，例如 `<| end_of_text|>` 或 `</answer>`（在我们的 `r1_zero` 提示中）。

4.1.2 轨迹

在强化学习中，我们从某个起始分布 $s_0 \sim \rho$ 得出的初始状态开始。然后，我们从策略 π_θ 中采样操作 a_t ，根据某个下一个状态分布 $s_{t+1} \sim \text{next_state}(s_t, a_t)$ 转换到下一个状态，然后重复。这一系列的状态和动作形成一个（有限视野）*trajectory*：

$$\tau = (s_0, a_0, s_1, a_1, \dots, s_T, a_T), \quad (3)$$

其中 T 是轨迹的长度，即 a_T 是文本结束标记，或者我们已达到标记的最大生成预算。

在我们的例子中， $s_0 \sim \rho$ 是我们的提示 x ，包含我们想要解决的数学问题（可能使用 CoT 提示或应用了少样本）。从这个前缀开始，我们逐个采样令牌，下一个状态是旧前缀与发出的令牌连接，即 $s_{t+1} = (s_t, a_t)$ 。轨迹也称为 *episodes* 或 *rollouts*；我们将互换使用这些术语。

4.1.3 奖励与回报

在强化学习中，标量奖励 $r_t = r(s_t, a_t)$ 判断在状态 s_t 下采取的操作的即时质量。对于经过验证的领域上的强化学习，就像我们的数学应用题任务一样，我们观察到中间推理步骤的奖励为零，直到我们输出答案。然后我们在终端操作上收到一个 *verified reward*

$$r_T = r(s_T, a_T) := \begin{cases} 1 & \text{if trajectory } \tau \text{ matches the ground-truth according to our reward function} \\ 0 & \text{otherwise} \end{cases}. \quad (4)$$

在我们的例子中，如果最终答案正确，则 r_T 为 1，否则为 0。*return* $R(\tau)$ 沿着轨迹聚合奖励，在我们的例子中只是 r_T 。

代理人的目标是最大化期望回报

$$J_\theta = E_{\tau \sim \pi_\theta}[r(\tau)], \quad (5)$$

其中 $\tau \sim \pi_\theta$ 是我们首先对 $s_0 \sim \rho$ 进行采样的轨迹上的分布，然后我们对策略中的操作和下一个状态分布中的下一个状态进行采样。在我们的例子中，这意味着首先从数据集中对数学问题 $x \sim \rho$ 进行采样，然后对标记进行采样，直到我们到达响应的末尾，这个过程可以用 $y \sim \pi_\theta(y | x)$ 表示。这个目标导致了优化问题

$$\theta^* = \arg \max_{\theta} J_\theta. \quad (6)$$

4.1.4 政策梯度

现在我们有学习名为 *policy gradients* 的 RL 算法所需的符号和定义。在作业的其余部分中，我们将坚持使用表示提示和响应的语言模型符号 (x, y) ，而不是状态 s_t 和动作 a_t 的 RL 符号。

回想一下，我们想要优化模型的预期奖励（即准确性），由下式给出

$$J_\theta = E_{x \sim \rho} E_{y \sim \pi_\theta(y | x)}[r(y | x)], \quad (7)$$

其中 $r(y | x)$ 表示响应 y 对于提示 x 是否正确。我们的方法是对目标进行梯度上升，或者

$$\theta_{k+1} = \theta_k + \alpha \nabla_{\theta} J_{\theta_k}, \quad (8)$$

但要做到这一点，我们需要能够估计样本的梯度 $\nabla_{\theta} J_{\theta_k}$ 。将此梯度重写为对样本的期望将产生一个称为“REINFORCE 策略梯度”的表达式，以 REINFORCE 论文 [R. J. 威廉姆斯, 1992]。特别是，我们将看到我们可以重写梯度如下：

$$\nabla_{\theta} E_{x \sim \rho} E_{y \sim \pi_\theta(y | x)}[r(y | x)] = E_{x \sim \rho} E_{y \sim \pi_\theta(y | x)}[r(y | x) \nabla_{\theta} \log \pi_\theta(y | x)]. \quad (9)$$

在进行推导之前，请注意该表达式意味着一种直观的算法（REINFORCE）：我们可以从数据集中采样问题 x ，从模型中采样响应 y ，

计算这些样本的 $r(y | x) \nabla_{\theta} \log \pi_\theta(y | x)$ 平均值，然后采取梯度步骤。表达式 $r(y | x) \nabla_{\theta} \log \pi_\theta(y | x)$ 表示我们应该增加具有高奖励 $r(y | x)$ 的响应 y 的对数概率。如果我们重复这个过程并采取多个梯度步骤，我们就有了一个基本的强化学习训练循环。

为了导出 REINFORCE 策略梯度，我们可以使用对数导数技巧（回想一下 $\log f(x)$ 的导数是 $f'(x)/f(x)$ ）：

$$\nabla_{\theta} \pi_\theta(y | x) = \pi_\theta(y | x) \nabla_{\theta} \log \pi_\theta(y | x). \quad (10)$$

直接应用此恒等式会产生所需的结果：

$$\nabla_{\theta} E_{y \sim \pi_\theta(y | x)}[r(y | x)] = \sum_y \nabla_{\theta} \pi_\theta(y | x) r(y | x) \quad (11)$$

$$= \sum_y \pi_\theta(y | x) \nabla_{\theta} \log \pi_\theta(y | x) r(y | x) \quad (12)$$

$$= E_{y \sim \pi_\theta(y | x)}[r(y | x) \nabla_{\theta} \log \pi_\theta(y | x)]. \quad (13)$$

请注意，在这些等式中，我们省略了提示 $E_{x \sim \rho}$ 的期望，因为 ∇_{θ} 和 $E_{x \sim \rho}$ 可以互换。

因此，给定一批提示 $x^{(1)}, \dots, x^{(B)} \stackrel{\text{iid}}{\sim} \rho$ 和每个提示 $y^{(i,j)} \stackrel{\text{iid}}{\sim} \pi_\theta(y | x^{(i)})$ 的 G 响应，我们有以下策略梯度估计样本：

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G r(y^{(i,j)} | x^{(i)}) \nabla_\theta \log \pi_\theta(y^{(i,j)} | x^{(i)}), \quad (14)$$

这会增加高奖励的响应。为了导出 GRPO，我们将从这个基本策略梯度估计器开始，然后一一应用修改。

4.1.5 基线

虽然基本的 REINFORCE 策略梯度估计器具有正确的期望 $\nabla_\theta J_\theta$ ，但它具有高方差，使得 RL 不稳定。为了稳定训练并减少方差，人们使用的一种工具是 *baselines*。

虽然基线可能变得相当复杂，但最简单的基线是从奖励中减去常数 b ：

$$E_{x \sim \rho} E_{y \sim \pi_\theta(y | x)} [(r(y | x) - b) \nabla_\theta \log \pi_\theta(y | x)]. \quad (15)$$

例如，如果我们减去 $b = 0.5$ 并且我们的奖励 r 是二元的（1 表示正确，0 表示错误），则使用此估计器意味着我们从模型中采样响应，增加正确响应的权重（权重为 0.5），并降低错误响应的权重（权重为 -0.5 ）。这与我们最初的梯度估计器形成对比，后者使用权重 1 增加正确响应的权重，并且对错误响应（使用权重 0）不执行任何操作。

本节的关键数学部分告诉我们，只要 b 不依赖于操作/响应 y （例如，它可以是常量、 x 的函数或策略的函数），减去基线 b 就可以保留估计器的期望。只要 b 不依赖于 y ，在我们的估计器中减去它就可以保留其期望：

$$E_{x \sim \rho} E_{y \sim \pi_\theta(y | x)} [(r(y | x) - b) \nabla_\theta \log \pi_\theta(y | x)] = E_{x \sim \rho} E_{y \sim \pi_\theta(y | x)} [r(y | x) \nabla_\theta \log \pi_\theta(y | x)]. \quad (16)$$

这一事实直接从以下恒等式得出：

$$E_{y \sim \pi_\theta(y | x)} [\nabla_\theta \log \pi_\theta(y | x)] = \nabla_\theta E_{y \sim \pi_\theta(y | x)} [1] = 0, \quad (17)$$

其中第一步是反向应用我们之前的对数导数技巧。

虽然基线保留了估计器的期望，但它们可以增加或减少方差。在符号中，“方差”的含义如下：将我们的策略梯度估计器写为样本均值 $\frac{1}{n} \sum_{i=1}^n Z_i$ ，其中每个 Z_i 是样本 (x, y) 的表达式 $(r(y | x) - b) \nabla_\theta \log \pi_\theta(y | x)$ ，我们的估计器的方差是 $\frac{E[Z_i^2] - E[Z_i]^2}{n}$ 。我们将探讨以下问题中基线何时增加或减少方差。

问题 (**baseline_calcs**)： 计算策略梯度估计器的方差（5 分）

让 π_θ 使用 $\pi_{\theta(A=1)} = p = \sigma(\theta)$ 定义二元动作空间 $\mathcal{A} = \{0, 1\}$ 上的策略，其中 σ 表示 sigmoid 函数 $\sigma(\theta) = \frac{1}{1+e^{-\theta}}$ 。我们的二元奖励函数会将奖励 1 分配给操作 $A = 1$ ，否则奖励 0，或 $r(A) = \mathbb{1}\{A = 1\}$ 。

(a) 让策略梯度估计器由下式给出

$$\frac{1}{n} \sum_{i=1}^n r(A_i) \nabla_\theta \log \pi_\theta(A_i) \quad (18)$$

给定 n 样本 $A_i \stackrel{\text{iid}}{\sim} \pi_{\theta^*}$ 。该估计量的方差是多少？

可交付成果：以 n 和 p 形式表达，并附有推导。

(b) 让基线调整后的策略梯度估计量由下式给出

$$\frac{1}{n} \sum_{i=1}^n (r(A_i) - b) \nabla_{\theta} \log \pi_{\theta}(A_i) \quad (19)$$

给定 n 样本 $A_i \stackrel{\text{iid}}{\sim} \pi_{\theta^*}$ 。该估计量的方差是多少？

可交付成果：用 n 、 b 和 p 进行表达，并附有推导和一些讨论。

(c) 如果我们替换“总体平均值”基线 $b = p$ ，结果方差是多少？将此方差与未调整的策略梯度估计器的方差进行比较：它是总是较低、总是较高，还是有时较高或较低，具体取决于 p ？

GRPO 中的基线

GRPO 的第一个组成部分是应用“组平均值”基线：

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G (r(y^{(i,j)} | x^{(i)}) - \mu_i) \nabla_{\theta} \log \pi_{\theta}(y^{(i,j)} | x^{(i)}) \quad (20)$$

对于 $\mu_i = \frac{1}{G} \sum_{j=1}^G r(y^{(i,j)} | x^{(i)})$ ，即提示 $x^{(i)}$ 时模型的平均奖励。请注意，尽管 μ_i 取决于响应 y ，但它仍然保留了重新调整 G^{-1} 之前的期望：

$$E_{y^{(1)}, \dots, y^{(G)} \sim \pi_{\theta}(y | x)} \left[\frac{1}{G} \sum_{j=1}^G (r(y^{(j)} | x) - \mu_i) \nabla_{\theta} \log \pi_{\theta}(y^{(j)} | x) \right] \quad (21)$$

$$= \frac{1}{G} \sum_{j=1}^G E_{y^{(2)}, \dots, y^{(G)} \sim \pi_{\theta}(y | x)} \left[E_{y^{(1)} \sim \pi_{\theta}(y | x)} [(r(y^{(1)} | x) - \mu_i) \nabla_{\theta} \log \pi_{\theta}(y^{(1)} | x)] \right] \quad (22)$$

$$= \frac{1}{G} \sum_{j=1}^G E_{y^{(1)} \sim \pi_{\theta}(y | x)} \left[\left(r(y^{(1)} | x) - \frac{1}{G} r(y^{(1)} | x) \right) \nabla_{\theta} \log \pi_{\theta}(y^{(1)} | x) \right] \quad (23)$$

$$= \frac{G-1}{G} E_{y \sim \pi_{\theta}(y | x)} [r(y | x) \nabla_{\theta} \log \pi_{\theta}(y | x)]. \quad (24)$$

这些按组均值调整后的奖励通常称为 *advantages*：该术语不是绝对奖励，而是表示此推出相对于平均奖励的“优势”。

4.1.6 优势标准化

GRPO 的下一个修改是在减去组平均值后除以组标准差：

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{r(y^{(i,j)} | x^{(i)}) - \mu_i}{\text{std}_i} \nabla_{\theta} \log \pi_{\theta}(y^{(i,j)} | x^{(i)}), \quad (25)$$

其中 $\text{std}_i = \sqrt{\frac{1}{G} \sum_{j=1}^G (r(y^{(i,j)} | x^{(i)}) - \mu_i)^2}$ （请注意，默认的 `torch.std` 函数使用略有不同的示例 `std` 表达式来进行偏差校正；您应该在实现中使用默认的 `torch.std`）。除以标准差不再保留估计器的期望，因此我们不再对原始目标 J_{θ} （预期奖励）进行梯度上升。考虑这种修改的一种方法是，它是一个稳定性技巧：如果我们假设每个梯度向量都是独立同分布的高斯分布，

除以组 std 可确保每个组的梯度更新在范数上大致相等。我们将在下一节中更深入地探讨优势标准化。

4.1.7 序列标准化

GRPO 中的另一个细节似乎继承自早期的 LLM-PPO 实现，称为 *sequence normalization*。回想一下，我们的策略梯度估计器（到目前为止）如下（我们将 $\log \pi_\theta(y^{(i,j)} | x^{(i)})$ 扩展为时间步长的总和）：

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{r(y^{(i,j)} | x^{(i)}) - \mu_i}{\text{std}_i} \nabla_\theta \sum_{t=1}^{\text{len}(y^{(i,j)})} \log \pi_\theta(y_t^{(i,j)} | x^{(i)}, y_{<t}^{(i,j)}). \quad (26)$$

GRPO 添加了一个额外的序列长度归一化项：

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{r(y^{(i,j)} | x^{(i)}) - \mu_i}{\text{std}_i} \nabla_\theta \left(\frac{1}{\text{len}(y^{(i,j)})} \sum_{t=1}^{\text{len}(y^{(i,j)})} \log \pi_\theta(y_t^{(i,j)} | x^{(i)}, y_{<t}^{(i,j)}) \right). \quad (27)$$

请注意，此修改进一步改变了期望：较长序列中的标记相对于较短序列中的标记被降低了权重，而不是在序列中平均加权所有标记。在下一节中，我们将讨论这个选择并进行消融，看看它是否是一个好的修改。但现在，我们将只实现标准 GRPO 算法，如原始论文中所述，因此我们将包括序列归一化。

4.1.8 将所有内容放在一起

我们还没有讨论 GRPO 的一个组成部分，即剪裁重要性重新加权。我们将在稍后的作业中讨论离策略强化学习时讨论这个组件。简而言之，离策略 RL 是指通过每个推理批次采取多个梯度步骤来加速 RL 的方法，而在策略 RL 中每个推理批次采取一个梯度步骤。术语“离策略”指的是这样一个事实：对于第二个梯度步骤，推理批次中的样本现在已经过时，因为它们来自旧版本的模型。重要性重新加权是对梯度估计器的修改，允许其使用陈旧的样本。目前，我们只会进行同策略强化学习，因此我们还不需要重要性重新加权。

我们现在拥有实施按策略 GRPO 所需的所有组件。该算法在算法 1 中提供。

Algorithm 1: On-policy Group Relative Policy Optimization (GRPO)

Input initial policy model π_{θ_0} ; reward function r ; task distribution (or dataset) ρ ; learning rate α

Output π_{θ}

1 policy model $\pi_{\theta} \leftarrow \pi_{\theta_0}$

2 For step = 1, ..., n_grpo_steps

3 Sample a batch of questions $x^{(1)}, \dots, x^{(B)} \stackrel{\text{iid}}{\sim} \rho$

4 Sample G outputs per question: $y^{(i,1)}, \dots, y^{(i,G)} \stackrel{\text{iid}}{\sim} \pi_{\theta}(y \mid x^{(i)})$

Compute the on-policy GRPO policy gradient estimator

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{1}{\text{len}(y^{(i,j)})} \sum_{t=1}^{\text{len}(y^{(i,j)})} \frac{r(y^{(i,j)} \mid x^{(i)}) - \mu_i}{\text{std}_i} \nabla_{\theta} \log \pi_{\theta}(y_t^{(i,j)} \mid x^{(i)}, y_{<t}^{(i,j)}), \quad (28)$$

where

$$\mu_i = \frac{1}{G} \sum_{j=1}^G r(y^{(i,j)} \mid x^{(i)}) \quad (29)$$

denotes the group mean and

$$\text{std}_i = \sqrt{\frac{1}{G} \sum_{j=1}^G (r(y^{(i,j)} \mid x^{(i)}) - \mu_i)^2} \quad (30)$$

denotes the group standard deviation.

Update θ with your favorite optimizer (we provide the stochastic gradient ascent update below):

$$\theta \leftarrow \theta + \alpha \hat{g}. \quad (31)$$

4.2 实施同保政策GRPO

4.2.1 使用拥抱脸部模型

虽然之前的作业在 cs336_basics 中使用了我们自己的语言模型实现，但在本次作业中，我们将直接使用 Hugging Face 转换器库来加载预训练的基础模型。如果您愿意，也欢迎您实现自己的变压器和预先训练的重量加载器；只需确保架构与 OLMo-2-0425-1B 的架构匹配即可。

要加载 Hugging Face 模型和分词器（在 bfloat16 中并使用 FlashAttention-2 来节省内存），您可以使用以下起始代码（在 cs336_alignment/checkpoint.py 中提供）：

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

def get_model_and_tokenizer(model_id_or_dir: str, device: str):
    model = AutoModelForCausalLM.from_pretrained(
        model_id_or_dir,
        device_map=device,
        torch_dtype=torch.bfloat16,
        attn_implementation="eager" if device=='cpu' else "flash_attention_2",
    )
```

```
tokenizer = AutoTokenizer.from_pretrained(model_id_or_dir)
return model, tokenizer
```

`model_id_or_directory`可以是模型名称（如 `allenai/OLMo-2-0425-1B`）或目录路径。该目录通常是调用 `save_pretrained` 的结果，您可以调用该目录来保存经过训练的模型：

```
# Save the model weights
model.save_pretrained(save_directory=output_dir)
tokenizer.save_pretrained(save_directory=output_dir)
```

首先，让我们实现一个辅助函数，它使用预先训练的分词器来对输入提示和响应进行分词。分别对提示和响应进行标记，而不添加特殊标记，然后将它们直接连接为 `prompt_ids + response_ids`，中间没有 EOS、BOS 或分隔符标记

他们。我们还需要构造一个 `response_mask`：一个布尔掩码，与移位后的 `labels` 对齐，即 `True` 正是标签标记来自响应的位置，而 `False` 则用于提示和填充位置。我们将在训练循环中使用这个掩码，以确保我们只计算响应令牌的损失。

问题（`tokenize_prompt_and_output`）： 提示和输出标记化（1分）

可交付成果：实现一个方法 `tokenize_prompt_and_output`，该方法将提示和输出分别标记，将它们连接在一起而不插入特殊标记，并构造一个 `response_mask`。推荐使用以下接口：

```
def tokenize_prompt_and_output(
    prompt_strs: list[str],
    output_strs: list[str],
    tokenizer: PreTrainedTokenizer,
) -> dict[str, torch.Tensor]:
```

对提示和输出字符串进行标记，并构造一个与 `labels` 对齐的掩码，其中 1 表示响应标记，0 表示其他标记（提示或填充）。

参数：

- **`prompt_strs: list[str]`**提示字符串列表。
- **`output_strs: list[str]`**输出字符串列表。
- **`tokenizer: PreTrainedTokenizer`**用于标记化的标记器。

返回：

- **`dict[str, torch.Tensor]`**.令 `prompt_and_output_lens` 为包含连接的标记化提示和输出字符串的长度的列表。那么返回的字典应该有以下键：

`input_ids` `torch.Tensor`，形状为 `(batch_size, max(prompt_and_output_lens) - 1)`：标记化的提示和输出字符串，最终标记被切掉。**`labels`** `torch.Tensor`，形状为 `(batch_size, max(prompt_and_output_lens) - 1)`：移位后的输入 id，即没有第一个标记的输入 id。

response_mask torch.Tensor, 形状为 (batch_size, max(prompt_and_output_lens) - 1): 与 labels 对齐的掩码, 当相应的标签标记是响应的一部分时, 值为 1, 否则为 0。

要测试您的代码, 请实施 `[adapters.run_tokenize_prompt_and_output]`。然后, 使用 `uv run pytest -k test_tokenize_prompt_and_output` 运行测试并确保您的实现通过它。

一旦我们有了标记化的输入, 我们就可以将其传递到模型中, 如下所示:

```
input_ids = train_batch["input_ids"].to(device)
labels = train_batch["labels"].to(device)
logits = model(input_ids).logits
```

有了这个语法, 请实现以下方法, 该方法计算模型下响应的每个令牌对数概率, 这是我们计算策略梯度所需的原语。在 RL 中, 记录每个标记的熵通常也很有用, 因此该函数将包含您也需要实现的 `return_token_entropy` 选项。

问题 (**get_response_log_probs**): 响应对数概率 (和熵) (1 分)

可交付成果: 实现一个方法 `get_response_log_probs`, 该方法从因果语言模型中获取每个标记的条件对数概率 (给定先前的标记), 以及可选地获取模型的下一个标记分布的熵。

推荐使用以下接口:

```
def get_response_log_probs(
    model: PreTrainedModel,
    input_ids: torch.Tensor,
    labels: torch.Tensor,
    return_token_entropy: bool = False,
) -> dict[str, torch.Tensor]:
```

参数:

- **model:** `PreTrainedModelHuggingFace` 模型用于评分 (如果不应计算梯度, 则放置在正确的设备上并处于推理模式)。
- **input_ids:** `torch.Tensor` 形状 (batch_size, sequence_length), 由标记化方法生成的串联提示 + 响应标记。
- **labels:** `torch.Tensor` 形状 (batch_size, sequence_length), 由标记化方法生成的标签。
- **return_token_entropy:** `bool` 如果 `True`, 还返回每个令牌的熵。

返回:

- `dict[str, torch.Tensor]`.
 - "log_probs" 形状 (batch_size, sequence_length), 条件对数概率 $\log p_{\theta}(x_t | x_{<t})$ 。
 - "token_entropy" 可选, 形状 (batch_size, sequence_length), 每个位置的每个标记熵 (仅在 `return_token_entropy=True` 时出现)。

要测试您的代码，请实施 `[adapters.run_get_response_log_probs]`。然后运行 `uv run pytest -k test_get_response_log_probs` 并确保测试通过。

4.2.2 在强化学习循环中使用 vLLM

在我们的 RL 循环中，我们还需要生成 rollout。我们将使用的具体配置是将 Hugging Face 模型和优化器放在一个 GPU 上进行训练，并将 vLLM（包括模型和 kv 缓存）放在另一个 GPU 上。因此，除了上一节中描述的 vLLM 初始化和生成函数之外，我们还需要在每个推理步骤之前同步两个设备之间的权重。我们在下面描述了权重同步代码，该代码在 `cs336_alignment/vllm_utils.py` 中可用。

```
@dataclass
class VLLMServer:
    gpu: int = 1 # Run training on gpu 0 and inference on gpu 1

    # Create the NCCL weight-transfer group between the training GPU and vLLM.
    def init_weight_sync(self, policy_device: str): ...

    # Copy the current Hugging Face policy weights into the vLLM server and
    # reset vLLM caches that depended on the old weights.
    def sync_policy_weights(self, policy: torch.nn.Module) -> None: ...

    # Generate rollouts from the current vLLM weights.
    def generate_completions(
        self,
        prompts: list[str],
        sampling_params: dict,
        batch_size: int | None = None,
    ) -> list[VLLMCompletion]: ...
```

与我们的提示实验一样，我们将使用温度 1.0、top-p 1.0、最大生成长度 512 进行采样。提示要求模型以字符串 `</answer>` 结束其答案，因此我们可以指示 vLLM 在模型输出此字符串时停止：

```
# Based on Dr. GRPO: stop when the model completes its answer
# https://github.com/sail-sg/understand-r1-zero/blob/
# c18804602b85da9e88b4aeeb6c43e2f08c594fbc/train_zero_math.py#L167
sampling_params['stop'] = ["</answer>"]
sampling_params['include_stop_str_in_output'] = True
```

4.2.3 GRPO 组件

接下来，让我们实现计算部分 GRPO 损失的组件。请注意，稍后在作业中，我们将实现变体，例如不同的优势标准化器或重要性重新加权方法。下面的组件经过精心设计，以便我们可以以最小的代码重叠在变体之间进行交换，因此我们提供了带有指定哪个变体的参数的接口。对于这部分作业，您只需实现标准 GRPO 变体。

我们的第一步是实现一个计算响应奖励的辅助函数。

Problem (compute_rollout_rewards): Computing the rewards of rollouts (1 point)

可交付成果：实现一种方法 `compute_rollout_rewards`，计算每个推出响应的原始奖励。

推荐使用以下接口：

```
def compute_rollout_rewards(
    reward_fn: Callable[[str, str], dict[str, float]],
    rollout_responses: list[str],
    repeated_ground_truths: list[str],
) -> tuple[torch.Tensor, dict[str, float]]:
```

计算推出响应列表的奖励以及奖励组件的元数据。参数：

- **reward_fn: Callable[[str, str], dict[str, float]]** 根据基本事实对推出响应进行评分，生成带有键 "reward"、"format_reward" 和 "answer_reward" 的字典。
- **rollout_responses: list[str]** 从政策中推出。该列表的长度是 `rollout_batch_size = n_prompts_per_rollout_batch * group_size`。
- **repeated_ground_truths: list[str]** 示例的基本事实。该列表的长度为 `rollout_batch_size`，因为每个示例的基本事实都重复 `group_size` 次。

返回：

- **tuple[torch.Tensor, dict[str, float]]**.
 raw_rewards 形状 `(rollout_batch_size,)`。每个推出响应的非标准化奖励。 **metadata** 记录奖励统计数据。至少包括整个推出批次的平均总奖励和格式奖励。

要测试您的代码，请实施 `[adapters.run_compute_rollout_rewards]`。然后，使用 `uv run pytest -k compute_rollout_rewards` 运行测试并确保您的实现通过它。

接下来，让我们在 GRPO 中实现核心数学，将这些奖励标准化，将其转化为优势。请注意，奖励（上面函数的输出）是扁平化的，因此我们需要考虑组大小，将其重新整形回组以进行标准化。要通过测试，您需要支持 `baseline = "mean"` 和 `advantage_normalizer = "std"`。为了避免被零除，您还应该在除法之前将 `advantage_eps` 添加到每组 `std` 中。

问题 (`compute_group_normalized_rewards_grpo`): 组标准化 (1 分)

可交付成果：实现一种方法 `compute_group_normalized_rewards`，将其组内的原始奖励标准化，并返回标准化奖励以及您认为有用的任何元数据。

目前，您只需要支持 `baseline = "mean"` 和 `advantage_normalizer = "std"`。对于不支持的输入，请随意提出 `NotImplementedError`。在作业的后面部分，我们将实现其他选项并运行消融。请记住将 `advantage_eps` 添加到标准化器以避免被零除。

推荐使用以下接口：

```
def compute_group_normalized_rewards(
    raw_rewards: torch.Tensor,
    group_size: int,
    baseline: Literal["mean", "none"] = "mean",
    advantage_eps: float = 1e-6,
    advantage_normalizer: Literal["std", "none", "mean"] = "std",
):
```

通过在每个组内应用所请求的基线和标准化来计算优势。

参数:

- **raw_rewards:** `torch.Tensor`形状(rollout_batch_size,)。每个推出响应的非标准化奖励，其中 rollout_batch_size = n_prompts_per_rollout_batch * group_size。
- **group_size:** `int`每个问题（组）的答复数量。
- **baseline:** `Literal["mean", "none"]`对于这个问题，支持mean，它减去每组的平均奖励。稍后，none 将意味着没有基线减法。
- **advantage_eps:** `float`小常数以避免标准化时被零除。
- **advantage_normalizer:** `Literal["std", "none", "mean"]`对于这个问题，支持std，即除以每组标准差。稍后，none 将意味着没有标准化，而 mean 将意味着除以每组平均奖励。

返回:

- `tuple[torch.Tensor, dict[str, float]]`.
advantages 形状 (rollout_batch_size,)。每个推出响应的群体标准化奖励。
metadata 您选择要记录的其他统计数据（例如奖励的平均值、标准差、最大/最小）。

要测试您的代码，请实施 `[adapters.run_compute_group_normalized_rewards]`。然后，使用 `uv run pytest -k compute_group_normalized_rewards_grpo` 运行测试并确保您的实现通过它。

下一步是实现一个计算每个令牌的策略梯度损失的函数。注意：算法 1 中的方程写出了梯度估计器，但是这个问题要求您编写每个令牌的损失，以便采用梯度产生GRPO梯度估计器的每一项（然后我们将在令牌上聚合）。这个表达式并不是传统意义上的损失，我们期望它在训练过程中下降。这只是一个表达式，当我们获取梯度时，我们就得到了策略梯度。

您应该实施的具体损失是

$$J_{\theta}^{\text{GRPO-on-policy}} = \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{1}{\text{len}(y^{(i,j)})} \sum_{t=1}^{\text{len}(y^{(i,j)})} \frac{r(y^{(i,j)} | x^{(i)}) - \mu_i}{\text{std}_i} \log \pi_{\theta}(y_t^{(i,j)} | x^{(i)}, y_{<t}^{(i,j)}), \quad (32)$$

您可以检查它在微分时会产生算法 1 中的梯度项。由于 `PyTorch` 优化器执行梯度下降，因此您的实现应该返回该目标的负值作为损失。 `compute_policy_gradient_loss` 将计算每个令牌和每个序列的损失项，而 `aggregate_loss_across_microbatch_sequence` 将计算令牌和序列中每个令牌损失的平均值。

问题 (`compute_policy_gradient_loss_on_policy`) : 在策略策略梯度 (1 分)

可交付成果: 实现一种方法 `compute_policy_gradient_loss`, 在给定原始奖励或预先计算的优势的情况下计算每个代币的策略梯度损失。

目前, 我们假设所有推出都是合规的, 因此您只需要支持 `importance_reweighting_method = "none"`, 并且可以忽略 `old-log-prob` 和 `Clipping` 参数。对于不支持的输入, 请随意提出 `NotImplementedError`。我们将在稍后的作业中实现裁剪。

推荐使用以下接口:

```
def compute_policy_gradient_loss(
    raw_rewards_or_advantages: torch.Tensor,
    policy_log_probs: torch.Tensor,
    importance_reweighting_method: Literal["none", "noclip", "grpo", "gspe"] = "none",
    old_log_probs: torch.Tensor | None = None,
    cliprange: float | None = None,
    response_mask: torch.Tensor | None = None,
) -> tuple[torch.Tensor, dict[str, torch.Tensor]]:
```

计算每个令牌的策略梯度损失, 其中 `raw_rewards_or_advantages` 是原始奖励或已经标准化的优势。

参数:

- **`raw_rewards_or_advantages: torch.Tensor`** 形状 `(batch_size,)` 或 `(batch_size, 1)`, 每个推出响应的标量奖励/优势。
- **`policy_log_probs: torch.Tensor`** 形状 `(batch_size, sequence_length)`, 每个标记的对数概率。
- **`importance_reweighting_method: Literal["none", "noclip", "grpo", "gspe"]`** `"none"`: 没有重要性重新加权; `"noclip"`: 应用重要性重新加权而不进行裁剪; `"grpo"`: 进行 PPO/GRPO 风格的令牌级别重新加权和裁剪; `"gspe"`: 进行 GSPO 风格的序列级重新加权和剪辑。
- **`old_log_probs: torch.Tensor | None`** 必需, 除非 `importance_reweighting_method = "none"`; 形状 `(batch_size, sequence_length)`。
- **`cliprange: float | None = None`** 剪辑参数 ϵ , 当 `importance_reweighting_method` 为 `"grpo"` 或 `"gspe"` 时需要。
- **`response_mask: torch.Tensor | None = None`** 响应标记上的可选形状 `(batch_size, sequence_length)` 掩码。对于仅对响应标记求平均序列级对数比的 GSPO 实现是必需的。

返回:

- **`tuple[torch.Tensor, dict[str, torch.Tensor]]`**.
 `per_token_policy_gradient_loss` 形状 `(batch_size, sequence_length)`, 每个令牌的策略梯度损失 (将在训练循环中的批次和序列维度上聚合)。
 `metadata` 来自底层损失调用的统计信息, 例如剪辑分数组件。

要测试您的代码，请实施 `[adapters.run_compute_policy_gradient_loss]`。然后运行 `uv run pytest -k test_compute_policy_gradient_loss_on_policy` 并确保测试通过。

最后，让我们实现令牌和序列的聚合函数，在标准 GRPO 中，该函数涉及对每个序列中的令牌进行平均，然后对序列进行平均。稍后，我们将实现仅除以常数的替代方法，我们在推导中看到该方法更忠实于我们试图估计的原始策略梯度。

问题 (`aggregate_loss_across_microbatch_sequence`): 令牌和序列之间的聚合损失 (0.5 分)

可交付成果：实现一种方法 `aggregate_loss_across_microbatch`，该方法接受每个令牌策略梯度损失和响应掩码，并输出平均损失。现在，我们将使用标准 GRPO 聚合，其中首先对每个序列进行平均，然后对序列进行平均，因此您可以假设 `loss_normalization = "sequence"`。对于不支持的输入，请随意提出 `NotImplementedError`。

推荐使用以下接口：

```
def aggregate_loss_across_microbatch(
    per_token_policy_gradient_loss: torch.Tensor,
    mask: torch.Tensor,
    loss_normalization: Literal["sequence", "constant"] = "sequence",
    normalization_constant: int | None = None,
) -> torch.Tensor:
```

根据响应掩码和损失归一化策略聚合每个令牌的策略梯度损失。

参数：

- **per_token_policy_gradient_loss**: `torch.Tensor` 形状 `(batch_size, sequence_length)`，每个令牌的策略梯度损失（将在训练循环中的批次和序列维度上聚合）。
- **mask**: `torch.Tensor` 形状为 `(batch_size, sequence_length)`，表示哪些位置应包含在损失中。
- **loss_normalization**: `Literal["sequence", "constant"] = "sequence"` "sequence": 每个序列的平均损失，然后对序列进行平均； "constant": 将总损失归一化为常数。
- **normalization_constant**: `int | None = None` 总损失除以的常数；如果 `loss_normalization = "constant"` 则为必需。

返回：

- **loss**: `torch.Tensor` 包含平均损失的标量。确保您稍后可以追回此损失。

要测试您的代码，请实施 `[adapters.run_aggregate_loss_across_microbatch]`。然后运行 `uv run pytest -k test_aggregate_loss_across_microbatch_sequence` 并确保测试通过。

4.2.4 GRPO训练步骤

我们现在准备将这些组件组合成一个完整的训练步骤。

梯度累积

我们需要较大的批量大小才能在推理过程中获得良好的利用率。因此，在同策略 RL 中，我们将训练批次大小设置为等于推理批次大小，我们的 GPU 将没有足够的内存来同时计算整个批次的梯度。因此，我们需要将批次分成一系列 *microbatches* 并累积这些微批次的梯度。主要棘手的部分是正确处理归一化，以确保微批次累积的梯度相当于计算整个批次的梯度。

梯度累积在 PyTorch 中实现起来很简单。回想一下，每个权重张量都有一个存储其梯度的属性 `.grad`。在我们调用 `loss.backward()` 之前，`.grad` 属性为 `None`。在我们调用 `loss.backward()` 之后，`.grad` 属性包含梯度。通常，我们会采取优化器步骤，然后使用 `optimizer.zero_grad()` 将梯度归零，这会重置权重张量的 `.grad` 字段：

```
# Forward pass.
logits = model(inputs)
loss = loss_fn(logits, labels)

# Backward pass.
loss.backward()

# Update weights.
optimizer.step()
# Zero gradients in preparation for next iteration.
optimizer.zero_grad()
```

为了实现梯度累积，我们将批次分成 k 微批次，并且只调用 `optimizer.step()` 和 `optimizer.zero_grad()` 一次。假设序列归一化，我们首先计算每个序列的平均损失，然后计算跨序列的平均损失，在计算平均微批次损失后，我们还需要根据序列数量重新加权。

```
gradient_accumulation_steps = 4
microbatch_size = len(inputs) // gradient_accumulation_steps
for i in range(0, len(inputs), microbatch_size):
    inputs_microbatch = inputs[i:i+microbatch_size]
    labels_microbatch = labels[i:i+microbatch_size]

    # Forward pass.
    logits = model(inputs_microbatch)
    loss = loss_fn(logits, labels_microbatch) * (len(inputs_microbatch) / len(inputs))

    # Backward pass.
    loss.backward()
# Update weights once across entire batch.
optimizer.step()
# Zero gradients once across entire batch.
optimizer.zero_grad()
```

请注意，稍后当我们进行常量标准化而不是序列标准化时，微批次代码将略有变化以考虑不同的标准化。

实施训练步骤

在下面的函数中，我们最终将实现 GRPO 训练步骤，并进行一些批次的部署。该函数将采用模型、分词器、优化器、奖励函数、提示和推出以及各种超参数，并且它将累积梯度并采取优化器步骤。您将需要实现如上所述的梯度累积，以免耗尽内存。在优化器步骤之前，函数还应将梯度范数剪辑为 `max_grad_norm`。然后，该函数应返回批次的训练损失以及元数据，两者均用于记录。请在训练步骤中至少记录以下指标：

- 损失。
- 梯度范数。
- 令牌熵。
- 训练奖励（总数，格式）。

我们也欢迎您记录其他指标：例如，在 1 和 `group_size` 之间记录 `k` 的 `pass@k` 可能会很有用（即在第一个 `k` 推出中至少有一个正确答案的提示分数）。

问题 (`grpo_train_step_standard_on_policy`): GRPO 训练步骤 (5 分)

可交付成果：在给定模型、分词器和部署的情况下，实现策略梯度的单批更新。对于这部分作业，您只需在 on-policy 设置中为标准 GRPO 或 `baseline = "mean"`、`advantage_normalizer = "std"`、`importance_reweighting_method = "none"` 和 `loss_normalization = "sequence"` 实现此函数。对于不支持的输入，请随意提出 `NotImplementedError`。

推荐使用以下接口：

```
def grpo_train_step(
    model: PreTrainedModel,
    tokenizer: PreTrainedTokenizer,
    optimizer: Optimizer,
    gradient_accumulation_steps: int,
    max_grad_norm: float | None,
    reward_fn: Callable[[str, str], dict[str, float]],
    repeated_prompts: list[str],
    rollout_responses: list[str],
    repeated_ground_truths: list[str],
    group_size: int,
    # Reward normalization
    baseline: Literal["mean", "none"] = "mean",
    advantage_eps: float = 1e-6,
    advantage_normalizer: Literal["std", "none", "mean"] = "std",
    # Importance reweighting and clipping
    importance_reweighting_method: Literal["none", "noclip", "grpo", "gspe"] = "none",
    old_log_probs: torch.Tensor | None = None,
    cliprange: float | None = None,
    # Loss normalization
    loss_normalization: Literal["sequence", "constant"] = "sequence",
    normalization_constant: int | None = None,
) -> tuple[torch.Tensor, dict[str, torch.Tensor | float]]:
```

使用 `gradient_accumulation_steps` 微批次执行前向和后向传递。

参数:

- **model:** `PreTrainedModel` 要训练的 HuggingFace 模型。
- **tokenizer:** `PreTrainedTokenizer` 用于标记化的标记器。
- **optimizer:** `Optimizer` 模型的优化器。
- **gradient_accumulation_steps:** `int` 每个优化器步骤的微批次数。
- **max_grad_norm:** `float | None` 如果不是 `None`, 则在调用 `optimizer.step()` 之前将梯度范数剪裁为此值。
- **reward_fn:** `Callable[[str, str], dict[str, float]]` 根据基本事实对推出响应进行评分, 生成带有键 "reward"、"format_reward" 和 "answer_reward" 的字典。
- **repeated_prompts:** `list[str]` 示例的提示。该列表的长度为 `rollout_batch_size`, 因为每个示例的提示都重复 `group_size` 次。
- **rollout_responses:** `list[str]` 从政策中推出。该列表的长度是 `rollout_batch_size = n_prompts_per_rollout_batch * group_size`。
- **repeated_ground_truths:** `list[str]` 示例的基本事实。该列表的长度为 `rollout_batch_size`, 因为每个示例的基本事实都重复 `group_size` 次。
- **group_size:** `int` 每个问题 (组) 的答复数量。
- **baseline:** `Literal["mean", "none"]` 如果 `mean`, 减去每组平均奖励; 如果是 `none`, 则不执行任何操作。
- **advantage_eps:** `float` 小常数以避免标准化时被零除。
- **advantage_normalizer:** `Literal["std", "none", "mean"]` 如果 `std`, 除以每组标准差; 如果 `none`, 则不执行任何操作; 如果 `mean`, 除以每组平均奖励。
- **importance_reweighting_method:** `Literal["none", "noclip", "grpo", "gspo"]` "none": 没有重要性重新加权; "noclip": 应用重要性重新加权而不进行裁剪; "grpo": 进行 PPO/GRPO 风格的令牌级别重新加权和裁剪; "gspo": 进行 GSPO 风格的序列级重新加权和剪辑。
- **old_log_probs:** `torch.Tensor | None` 必需, 除非 `importance_reweighting_method = "none"`; 形状 `(batch_size, sequence_length)`。
- **cliprange:** `float | None = None` 剪辑参数 ϵ , 当 `importance_reweighting_method` 为 "grpo" 或 "gspo" 时需要。
- **loss_normalization:** `Literal["sequence", "constant"] = "sequence"` "sequence": 平均损失对每个序列进行平均; "constant": 将总损失归一化为常数 (对于所有训练都是固定的)。
- **normalization_constant:** `int | None = None` 总损失除以的常数; 如果是 `loss_normalization = "constant"`, 则为必需。

返回:

- **tuple[torch.Tensor, dict[str, torch.Tensor]].**
损失标量张量。批次损失, 根据梯度累积进行调整。我们返回这个以便我们可以记录它。
元数据 Dict, 包含来自底层损失调用的元数据、裁剪前的梯度范数以及您可能想要记录的任何其他统计数据。

要测试您的代码，请实施 `[adapters.run_grpo_train_step]`。然后运行 `uv run pytest -k test_grpo_train_step_standard_on_policy` 并确认通过。

4.3 实验

我们现在准备构建完整的 GRPO 训练循环，它应该：

- 加载模型和数据集
- 初始化日志记录（例如 Wandb）、vLLM 服务器、优化器
- 执行 RL 训练循环

我们在下面提供了一些建议的超参数。如果您的训练脚本正确，对于大多数随机种子，在使用这些超参数进行训练时，您应该会看到验证奖励随着时间的推移而增加。

```
n_train_examples = 6400
n_val_examples = 1024
num_rollout_steps = 200
learning_rate = 1e-5
rollout_batch_size = train_batch_size = 256
group_size = 8
gradient_accumulation_steps = 32
sampling_temperature = 1.0
sampling_max_tokens = 512
max_grad_norm = 1.0
optimizer = torch.optim.AdamW(
    policy.parameters(), lr=learning_rate, betas=(0.9, 0.95), weight_decay=0.0
)
```

请注意，`rollout_batch_size` 和 `train_batch_size` 计算的是响应，而不是提示。因此，`rollout_batch_size = train_batch_size = 256` 意味着 32 会提示每个 8 卷展。

作为合理的默认值，您可以每 10 个推出批次对验证集进行一次评估。您应该确保对至少 1024 个验证示例进行评估，因为 CoT/RL 评估可能会有噪音。除了记录上一小节中描述的指标之外，记录推出也很有用，可以定性地了解模型正在做什么。合理的默认设置是每 40 个部署批次记录当前批次部署的训练部署。请随意使用 `data/gsm8k/train.jsonl` 作为训练示例，使用 `data/gsm8k/test.jsonl` 作为验证示例。

由于策略梯度估计器可能具有高方差，并且 RL 是自我强化的，因此 RL 训练运行轨迹往往具有非常高的方差。因此，一旦您确信脚本正确，我们将要求您使用 4 随机种子运行 RL 循环。如果您的脚本在给定固定种子的情况下不能完全重现，那也没关系；您也可以只运行实验 4 次。在您的图中，请绘制指标并显示运行间方差的一些指示（即不要只绘制平均值）。一些合理的选择包括：

- 绘制 4 运行的平均值，并直接绘制实际运行（为了便于阅读，它可能帮助将实际运行放置在单独的绘图上，或使用较低的不透明度、较小的线条粗细或虚线）
- 绘制平均值以及阴影置信区间：具体来说，对于每个步骤，计算样本平均值和样本标准差。合理的间隔是 $\mu \pm 1.96 * \frac{\sigma}{\sqrt{n}}$ ，其中 n 是种子数。

- 绘制每一步的平均值以及最小值和最大值。

问题 (`grpo_experiments_standard_on_policy`): 使用 GRPO 提高 GSM8K 上的 OLMo-2-0425-1B 性能 (2 B200 小时) (10 分)

(a) 编写一个脚本, 在给定模型名称、提示、训练集文件路径、验证集文件路径、采样超参数和训练超参数的情况下运行 GRPO 训练循环。在较高级别上, 该脚本将首先涉及初始化 vLLM 服务器、wandb 日志记录、数据集、模型和优化器。然后, 训练循环涉及重复以下操作: 与 vLLM 服务器同步权重、生成训练部署、对训练批次采取策略梯度以及定期检查验证集的性能 (同时记录生成)。

可交付成果: 在任务 GSM8K 和模型 OLMo-2-0425-1B 上运行 GRPO (标准、符合策略) 的脚本。

b) 运行脚本大约 50 个步骤, 并确认您看到验证奖励有所改善, 并且随着时间的推移进行合理的部署。注意: 奖励一开始接近于零是正常的, 并且根据随机种子, 在我们对非零奖励的推出进行采样并且模型开始改进之前可能需要执行几个步骤。

可交付成果: 使您相信您的脚本是正确的证据 (例如, 改进每个步骤的验证奖励、看起来合理的部署)。

(c) 一旦您确信脚本是正确的, 请使用上面提供的超参数运行它, 并使用 4 随机种子, 在 GSM8K 上运行 OLMo-2-0425-1B。您应该使用零样本 `r1_zero` 提示符。如果代码正确, 您应该会看到验证奖励随着模型的训练而提高。

记录以下指标, 并随时间绘制它们的图, 同时注意运行之间的差异 (例如, 通过绘制运行中的平均值/标准差或最小/最大值)。

- 损失。
- 梯度范数。
- 令牌熵。
- 训练奖励 (总数, 格式)。
- Val 奖励 (总计, 格式)。
- Val 平均响应长度。
- 您认为对调试有用的任何其他内容。

还请定期记录并观察部署情况。培训后反应是否有所改善?

可交付成果: 几句话评论和每个指标的绘图, 描述它在训练过程中如何变化以及运行之间有多少差异。

可交付成果: 训练前后模型展示的一些示例。

可交付成果: 最终验证准确度至少达到 25% (随机种子的平均值) 的过程。

Now that we have a working script, we can start tuning hyperparameters and doing ablations. We'll start by tuning the learning rate, which is the hyperparameter that typically has the largest effect on training.

问题 (grpo_learning_rate): 调整学习率 (4 B200 小时) (3 分)

从建议的超参数开始，对学习率进行扫描（至少一个学习率比建议的默认值小，一个学习率比建议的默认值大），并报告最终的验证奖励（如果优化器发散，则注意发散）。根据您在上一部分中观察到的方差量，您应该决定每次训练运行使用多少个随机种子。对于作业的其余部分，如果您愿意，可以随意使用调整后的学习率而不是建议的默认值。

可交付成果：最终验证奖励与学习率的关系图，以及几句评论。

影响 RL 动态的另一种选择是提示：因为 RL 涉及增加模型推出的权重，所以提示可以决定模型在训练期间探索哪些推出。我们将在下面探讨即时选择的效果。

问题 (grpo_prompt_ablation): 及时消融 (4 B200 小时) (3 分)

使用 question_only 提示符和 r1_zero_three_shot 提示符运行 GRPO 脚本，并使用一些随机种子。与上面的零样本 r1_zero 提示结果相比，这些提示的表现如何？哪些提示具有最佳的平均奖励和最低的方差？其他记录的指标是否存在其他系统差异？根据运行之间的差异，您对自己的发现有多大信心？

可交付成果：评论，由参考指标的图表支持。

5 种强化学习算法变体

GRPO 是强化学习语言模型的热门选择，但其算法选择在各种论文中都存在争议。在本次作业中，我们将学习这些选择背后的一些理论论据，并进行受控实验，以了解哪些算法是最好的（至少对于我们的模型和数据集组合而言）。本节将介绍 on-policy 设置，下一节将介绍 off-policy 算法。

5.1 GRPO博士

在 GRPO 的推导过程中，您可能已经注意到一系列选择，这意味着 GRPO 策略梯度估计器不再具有“正确”期望 $\nabla_{\theta} J_{\theta}$ ，其中 J_{θ} 是模型的预期奖励。这两个选择是标准差优势归一化和序列长度归一化。我们将考虑的第一个消融，在 GRPO 博士论文 [Z. Liu et al., 2025]，就是撤销这些选择：删除标准标准化，并将总损失除以一个常数，而不是首先按每个序列的长度标准化。将此常数归一化器表示为 Z ，则 Dr.GRPO 梯度估计器由下式给出

$$\hat{g} \leftarrow \frac{1}{Z} \sum_{i=1}^B \sum_{j=1}^G \sum_{t=1}^{\text{len}(y^{(i,j)})} (r(y^{(i,j)} | x^{(i)}) - \mu_i) \nabla_{\theta} \log \pi_{\theta}(y_t^{(i,j)} | x^{(i)}, y_{<t}^{(i,j)}), \quad (33)$$

其中 Z 通常设置为训练批量大小乘以最大生成长度（即 $Z = BGL$ ，其中 L 是最大生成长度（在我们的例子中为 512））。

问题 (**think_about_length_normalization**): 考虑长度归一化 (1 分)

在进行任何实验之前，请考虑按序列长度标准化每个序列与按相同常数标准化所有序列之间的差异。每种方法的优点和缺点是什么？是否有特定的设置或示例表明一种方法似乎更好？

可交付成果：几句话的讨论。

问题 (**compute_group_normalized_rewards_dgrpo**): GRPO 博士群体标准化 (0.5 分)

可交付成果：更新您的方法 `compute_group_normalized_rewards` 以支持 `advantage_normalizer = "none"`。虽然 GRPO 博士使用的是 `baseline = "mean"`，但针对这个问题，还请添加对 `baseline = "none"` 的支持。

要测试您的代码，请实施 `[adapters.run_compute_group_normalized_rewards]`。然后，使用 `uv run pytest -k compute_group_normalized_rewards_dgrpo` 运行测试并确保您的实现通过它。

问题 (**aggregate_loss_across_microbatch_constant**): GRPO 博士损失聚合 (0.5 分)

可交付成果：更新您的方法 `aggregate_loss_across_microbatch` 以支持 `loss_normalization = "constant"`。

要测试您的代码，请实施 `[adapters.run_aggregate_loss_across_microbatch]`。然后运行 `uv run pytest -k test_aggregate_loss_across_microbatch_constant` 并确保测试通过。

5.2 拒绝微调

我们将研究的下一个消融是一种更简单的算法，通常称为“拒绝微调”（RFT）或“专家迭代”（EI）。顾名思义，该算法涉及对一堆滚动进行采样，保留正确的滚动，然后对这些正确的滚动进行监督微调（即最小化下一个单词预测日志损失）。具体来说，我们的 RFT 梯度是

$$\hat{g} \leftarrow \nabla_{\theta} \left[\frac{1}{Z} \sum_{i=1}^B \sum_{j=1}^G \mathbb{1}\{r(y^{(i,j)} | x^{(i)}) = 1\} \sum_{t=1}^{\text{len}(y^{(i,j)})} \log \pi_{\theta}(y_t^{(i,j)} | x^{(i)}, y_{<t}^{(i,j)}) \right], \quad (34)$$

其中 Z 是一个常数标准化器（如 Dr.GRPO 中的），而 $\mathbb{1}\{r(y^{(i,j)} | x^{(i)}) = 1\}$ 是一个仅保留正确响应的指示函数。在下面的问题中，我们将思考 RFT 梯度是否实际上是策略梯度（即它是否实际上优化了预期奖励 J_{θ} ），以及它与 GRPO 的关系。和以前一样，当将其实现为 PyTorch 损失时，返回负目标，以便最小化损失对估计器执行梯度上升。

问题 (think_about_rft): 思考 RFT (2 分)

回想一下，RFT 目标（具有恒定标准化）由下式给出

$$J_\theta = \frac{1}{Z} \sum_x \sum_{j=1}^G \mathbb{1}\{r(y^{(j)} | x) = 1\} \log \pi_\theta(y^{(j)} | x), \quad (35)$$

其中 x 是提示，每个 $y^{(j)}$ 表示响应 $y^{(j)} \stackrel{\text{iid}}{\sim} \pi_\theta(\cdot | x)$ ， G 是我们从策略中采样的代数， Z 是我们的常数标准化器， r 是奖励函数。RFT 梯度由 $\nabla_\theta J_\theta$ 给出。

相反，在策略 Dr.GRPO 策略梯度估计器（即具有恒定标准化且无标准标准化的 GRPO）由下式给出

$$\frac{1}{Z} \sum_x \sum_{j=1}^G (r(y^{(j)} | x) - \mu) \nabla_\theta \log \pi_\theta(y^{(j)} | x), \quad (36)$$

其中 μ 表示组平均值 $\mu = \frac{1}{G} \sum_{j=1}^G r(y^{(j)} | x)$ 。假设我们的奖励函数 r 是二元的，请比较和对比这些目标。他们有同样的期望吗？您预计哪个方差更低？凭直觉，在某些情况下您会更喜欢其中一种吗？

可交付成果：几句话的讨论。

5.3 最大RL

最后，我们将探索一种称为最大似然强化学习的最新方法，也称为 MaxRL [F. Tajwar 等人, 2026]。MaxRL 不是像 GRPO 中那样通过组 std 来标准化优势，或者像 GRPO 博士那样通过什么都不标准化优势，而是建议除以组平均值。这种选择会产生以下梯度估计器：

$$\hat{g} \leftarrow \frac{1}{Z} \sum_{i=1}^B \sum_{j=1}^G \sum_{t=1}^{\text{len}(y^{(i,j)})} \frac{r(y^{(i,j)} | x^{(i)}) - \mu_i}{\mu_i} \nabla_\theta \log \pi_\theta(y_t^{(i,j)} | x^{(i)}, y_{<t}^{(i,j)}). \quad (37)$$

请注意，在原始 MaxRL 论文中，它们不是使用常量标准化器 Z ，而是除以批次中的标记总数 $\sum_{i=1}^B \sum_{j=1}^G \text{len}(y^{(i,j)})$ （现在与批次相关，而不是常量）。在此作业中，我们将使用常量归一化器，以便相对于其他基线而言移动部分更少，从而更容易隔离通过 μ_i 而不是 std_i 进行归一化的影响。

在下面的问题中，我们将做一些基本数学来探索不同优势标准化器的效果。直观上，除以平均奖励 μ_i 平均会增加更困难的提示的权重，与更简单的提示相比，这些提示更有可能具有更低的 μ_i 。我们可以通过定义预期奖励的重新加权版本来形式化这种直觉：

$$J_{\theta,w} = E_{x \sim p} [w(x, \text{stopgrad}(\pi_\theta)) E_{y \sim \pi_\theta} r(y | x)], \quad (38)$$

其中 $w(x, \text{stopgrad}(\pi_\theta))$ 重新加权提示 x ，例如基于当前策略在 x 上的预期奖励捕获的难度 $\eta(x) = E_{y \sim \pi_\theta} r(y | x)$ ，而 stopgrad 表示我们不通过重新加权函数获取梯度。在下面的问题中，我们将为此迄今为止看到的每个优势标准化器导出难度重新加权。

问题 (**derive_difficulty_reweightings**) : 导出由不同优势标准化器引起的难度重新加权 (6 分)

回想一下, 总的来说, 标准策略梯度由下式给出

$$\nabla_{\theta} J_{\theta} = \nabla_{\theta} E_{x \sim \rho} [E_{y \sim \pi_{\theta}} r(y | x)], \quad (39)$$

其中 ρ 是我们对提示 x 的提示分布, π_{θ} 表示我们的策略, $r(y | x)$ 表示 y 是否是对 x 的正确响应。在这个问题中, 我们将导出由 GRPO 变体优化的替代目标, 其中这些替代目标采用以下形式

$$\nabla_{\theta} J_{\theta, w} = \nabla_{\theta} E_{x \sim \rho} [w(x, \text{stopgrad}(\pi_{\theta})) E_{y \sim \pi_{\theta}} r(y | x)] \quad (40)$$

对于提示 w 的一些重新加权 (我们不通过 w 进行区分)。

(a) 回想一下 Dr. GRPO 政策梯度估计量由下式给出

$$E_{x \sim \rho} \left[\frac{1}{Z} \sum_{j=1}^G (r(y^{(j)} | x) - \mu) \nabla_{\theta} \log \pi_{\theta}(y^{(j)} | x) \right] \quad (41)$$

其中 μ 表示组平均值 $\mu = \frac{1}{G} \sum_{j=1}^G r(y^{(j)} | x)$ 。设置常量归一化器 $Z = G$ 并采用组大小 $G \rightarrow \infty$, 对于什么重加权函数 w , 该估计器相当于优化代理目标 $J_{\theta, w}$?

可交付成果: 以问题参数子集的形式表达, 并带有几句论证。

(b) 假设恒定标准化, GRPO 估计量与 Dr. GRPO 的不同之处在于它除以标准差:

$$E_{x \sim \rho} \left[\frac{1}{Z} \sum_{j=1}^G \frac{r(y^{(j)} | x) - \mu}{\text{std}} \nabla_{\theta} \log \pi_{\theta}(y^{(j)} | x) \right], \quad (42)$$

其中 std 表示组标准差。设置常量归一化器 $Z = G$ 并采用组大小 $G \rightarrow \infty$, 对于什么重新加权函数 w , 该估计器相当于优化代理目标 $J_{\theta, w}$?

可交付成果: 以问题参数子集的形式表达, 并带有几句论证。

(c) MaxRL 除以组平均值, 或者

$$E_{x \sim \rho} \left[\frac{1}{Z} \sum_{j=1}^G \frac{r(y^{(j)} | x) - \mu}{\mu} \nabla_{\theta} \log \pi_{\theta}(y^{(j)} | x) \right]. \quad (43)$$

设置常量归一化器 $Z = G$ 并采用组大小 $G \rightarrow \infty$, 对于什么重加权函数 w , 该估计器相当于优化代理目标 $J_{\theta, w}$?

可交付成果: 以问题参数子集的形式表达, 并带有几句论证。

问题 (**think_about_advantage_normalization**): 考虑优势标准化 (2 分)

在进行任何实验之前，请考虑通过组标准差、组平均值对组优势进行归一化或不进行优势归一化之间的区别。每种方法的优点和缺点是什么？是否有特定的设置或示例表明一种方法似乎更好？

可交付成果：几句话的讨论。

问题 (`compute_group_normalized_rewards_maxrl`)： MaxRL 组归一化 (0.5 分)

可交付成果：更新您的方法 `compute_group_normalized_rewards` 以支持 `advantage_normalizer = "mean"`。与 `advantage_normalizer = "std"` 一样，您应该将 `advantage_eps` 添加到标准化器以避免被零除。

要测试您的代码，请实施 [`adapters.run_compute_group_normalized_rewards`]。然后，使用 `uv run pytest -k compute_group_normalized_rewards_maxrl` 运行测试并确保您的实现通过它。

5.4 实验

我们现在准备进行实验来探索这些不同的策略梯度估计器。首先，我们需要更新 `grpo_train_step` 方法以支持这些变体，我们在上面实现了其组件。具体设置如下：

- `GRPO_constant`: `baseline = "mean"`、`advantage_normalizer = "std"`、`loss_normalization = "constant"`
- `Dr_GRPO`: `baseline = "mean"`、`advantage_normalizer = "none"`、`loss_normalization = "constant"`
- `RFT`: `baseline = "none"`、`advantage_normalizer = "none"`、`loss_normalization = "constant"`
- `MaxRL` (不断标准化): `baseline = "mean"`、`advantage_normalizer = "mean"`、`loss_normalization = "constant"`

为了加快训练速度，请注意，一旦我们计算了每个序列的归一化优势，优势为零的序列就不需要传递到模型中，因为它们对梯度的贡献为零。由于我们的奖励是二元的，因此在两种情况下序列的优势为零：

- 如果是 `baseline = "mean"`，则如果组中的序列都具有相同的奖励，那么它们的优势将为零。
- 如果 `baseline = "none"`，任何奖励为零的序列将具有零优势。

一旦我们修剪了零优势序列，我们还可以将 `gradient_accumulation_steps` 减少相同的因子（即我们可以保持 `microbatch_size` 相同）：例如，如果一半序列具有零优势，我们可以采用 $\frac{k}{2}$ 梯度累加步骤而不是 k 梯度累加步骤。这种优化对于 RFT 特别有用，因为它有相当多的零奖励序列。但是您应该小心数学和实现，以确保修剪版本计算出与未修剪版本相同的梯度。

问题 (`grpo_train_step_variants_on_policy`): GRPO 训练步骤变体 (2.5 分)

可交付成果：更新您的 `grpo_train_step` 方法以支持全系列的 on-policy 变体（仍使用 `importance_reweighting_method = "none"`）。这些选项包括 `baseline`：\、`Literal["mean", "none"]`、`advantage_normalizer`：`Literal["std", "none", "mean"]` 和 `loss_normalization`：`Literal["sequence", "constant"]`。此外，为了加快训练速度，您的方法应避免将零优势序列传递到模型中。

请注意，对于 `baseline = "none"`，不正确的样本获得零奖励，因此损失权重为零，因此无需将它们传递到模型中。您可以利用这一事实来调整实施并加快培训速度。

要测试您的代码，请实施 `[adapters.run_grpo_train_step]`。然后运行 `uv run pytest -k test_grpo_train_step_variants_on_policy` 并确认通过。

现在我们的训练步骤已经实施，我们可以运行实验来观察我们方法的性能，并确定它们是否优于标准 GRPO。您应该使用与之前的运行相同的超参数，以使您的运行具有可比性。

关于调整超参数的说明：如果我们的目标是确定一种方法是否优于另一种方法，那么最具实验性的合理方法是为每种方法单独调整超参数。然而，超参数调整的成本很高，而且我们没有那么多的计算能力来完成这项任务。另一种更便宜的选择是调整基线的超参数，然后将这些超参数用于新方法（例如，在我们的例子中，我们调整了标准 GRPO 的学习率，并在以下实验中使用该学习率）。在这种情况下，如果我们的新方法优于基线，我们仍然可以声称它更好，因为次优学习率会为新方法的正确调整性能产生下限。但另一方面，如果新方法表现更差，我们不能声称它比基线更差，因为我们没有正确调整其超参数。

问题 (`grpo_experiments_variants_on_policy`): 比较不同 RL 算法的性能（8 B200 小时）（10 分）

保持标准 GRPO 训练运行的超参数固定（根据您选择的学习率），运行以下变体，每个变体使用 4 随机种子。您应该使用零样本 `r1_zero` 提示符。

- `GRPO_constant`：标准 GRPO，但对损失聚合进行持续归一化，而不是序列归一化。
- `Dr_GRPO`：`GRPO_constant` 但与 `advantage_normalizer = "none"`。
- `RFT`：`GRPO_constant` 但带有 `advantage_normalizer = "none"` 和 `baseline = "none"`。请注意，对于此方法，不需要将错误的样本传递给正在训练的模型，这应该可以加快训练速度。
- `MaxRL`：`GRPO_constant` 但与 `advantage_normalizer = "mean"`。请注意，原始 MaxRL 实现通过每个微批次的标记数量对损失进行归一化，但我们将使用常量归一化。

与您之前运行的标准 GRPO 运行相比，这些方法的表现如何？哪些方法具有最好的平均性能，哪些方法具有最低的方差？您认为是否有一些方法可以从更多的超参数调整中受益？根据运行之间的差异，您对自己的发现有多大信心？

6 离策略强化学习

到目前为止，我们的实验和算法都是 *fully on-policy*，这意味着每个梯度估计器使用直接从正在更新的模型中抽取的样本（即，我们对每个推理批次进行 1 个训练步骤，并且将训练批次大小设置为等于推理批次大小）。在本节中，我们将探讨 *off-policy RL*，它每个推理批次需要多个训练步骤，有可能加快训练速度，但代价是潜在的不稳定性和增加的算法复杂性。

6.1 重要性重新加权

在策略梯度的普通版本中，我们对一批响应进行采样，然后对这些响应采取单个大批量梯度步骤。但是，如果我们将批次分成一系列小批次，然后每个小批次采取一个训练步骤，我们可能希望模型训练得更快。这种方法称为“离策略 RL”，与我们最初的“在策略 RL”方法相反，在“on-policy RL”中，我们每个推理批次执行一个训练步骤。

这种方法被称为离策略强化学习的原因是，在第一个小批量步骤之后，当前的策略现在与我们在推理过程中使用的策略不同。这些样本现在是“脱离策略”或“过时的”：它们不是来自当前策略，我们希望为其计算策略梯度。在符号中，如果我们将推理策略表示为 π_0 ，将当前策略表示为 π_θ ，则我们有以下结果：

$$E_{x \sim \rho} E_{y \sim \pi_0(y|x)} [r(y|x) \nabla_\theta \log \pi_\theta(y|x)] \neq E_{x \sim \rho} E_{y \sim \pi_\theta(y|x)} [r(y|x) \nabla_\theta \log \pi_\theta(y|x)]. \quad (44)$$

左侧是我们的“天真的”离策略估计器，我们假装响应 y 来自我们当前的策略，并机械地应用相同的步骤来计算我们的策略梯度。这个方程表明这个朴素的估计器没有正确的期望。

纠正这种偏差的一种方法是使用 *importance reweighting*，它会生成以下估计器：

$$E_{x \sim \rho} E_{y \sim \pi_0(y|x)} \left[\frac{\pi_\theta(y|x)}{\pi_0(y|x)} r(y|x) \nabla_\theta \log \pi_\theta(y|x) \right], \quad (45)$$

我们应用了 *importance weights* $\frac{\pi_\theta(y|x)}{\pi_0(y|x)}$ 。这些权重具有增加与当前策略 π_θ 仍然相关的 y 的权重，同时降低 $\pi_\theta(y|x)$ 较低的“过时”样本的权重的效果。不难看出这个修改后的表达式具有正确的期望：

$$E_{x \sim \rho} E_{y \sim \pi_0(y|x)} \left[\frac{\pi_\theta(y|x)}{\pi_0(y|x)} r(y|x) \nabla_\theta \log \pi_\theta(y|x) \right] = E_{x \sim \rho} E_{y \sim \pi_\theta(y|x)} [r(y|x) \nabla_\theta \log \pi_\theta(y|x)]. \quad (46)$$

缺点是重要性重新加权会增加估计器的方差：如果重要性权重 $\frac{\pi_\theta(y|x)}{\pi_0(y|x)}$ 可以与 C 一样大，那么我们的方差也会增加 C 倍。这个问题使得简单的重要性重新加权不适用于语言模型，其重要性权重是 $\text{len}(y)$ 项的乘积，使其响应长度呈指数增长：

$$\frac{\pi_\theta(y|x)}{\pi_0(y|x)} = \prod_{t=1}^{\text{len}(y)} \frac{\pi_\theta(y_t|x, y_{<t})}{\pi_0(y_t|x, y_{<t})}. \quad (47)$$

因此，与具有高偏差但没有因重新加权引入的额外方差的“朴素”未加权离策略估计器相比，该重新加权估计器具有零偏差但高方差。以下方法将位于中间的某个位置，提出各种启发式方法来驾驭这种偏差-方差权衡。

6.2 PPO/GRPO 式重要性重新加权和裁剪

6.2.1 代币级别重新加权

标准方法，首先在 PPO 中提出 [J. Schulman et al., 2017] 也被 GRPO 采用 [Z. Shao et al., 2024]，是进行标记级重要性重新加权而不是序列级重新加权。此选择会产生以下“未剪辑的令牌级别重新加权”估计器：

$$E_{x \sim \rho} E_{y \sim \pi_0(y|x)} \left[r(y|x) \sum_{t=1}^{\text{len}(y)} \frac{\pi_\theta(y_t | x, y_{<t})}{\pi_0(y_t | x, y_{<t})} \nabla_\theta \log \pi_\theta(y_t | x, y_{<t}) \right]. \quad (48)$$

请注意，这个表达式与我们上面看到的“有原则的”序列级估计器不同，它可以重写如下（随着时间步长扩展）：

$$E_{x \sim \rho} E_{y \sim \pi_0(y|x)} \left[r(y|x) \left(\prod_{t=1}^{\text{len}(y)} \frac{\pi_\theta(y_t | x, y_{<t})}{\pi_0(y_t | x, y_{<t})} \right) \sum_{t=1}^{\text{len}(y)} \nabla_\theta \log \pi_\theta(y_t | x, y_{<t}) \right]. \quad (49)$$

在深入了解此修改后的表达式的作用之前，请注意重新加权项在 $\text{len}(y)$ 中不再呈指数关系，从而大大减少了方差。

与作业的前面部分一样，为了更好地理解这个估计器，我们将推导出该估计器正在优化的代理目标 [T. Debris 等人, 2013]。具体来说，让 $\tilde{\pi}_t$ 表示代理策略，我们从旧策略 π_0 中对所有时间步骤进行采样，除了从 π_θ 中采样的步骤 t 之外：

$$\tilde{\pi}_t(y|x) = \left(\prod_{s=1}^{t-1} \pi_0(y_s | x, y_{<s}) \right) \pi_\theta(y_t | x, y_{<t}) \left(\prod_{s=t+1}^{\text{len}(y)} \pi_0(y_s | x, y_{<s}) \right). \quad (50)$$

然后，如果我们假设所有响应都具有相同的长度 L ，则事实证明，令牌级别重新加权优化了每个 $\tilde{\pi}_t$ 下的预期奖励，或者

$$J_\theta^{\text{token}} = E_x \left[\sum_{t=1}^L E_{y \sim \tilde{\pi}_t(y|x)} [r(y|x)] \right]. \quad (51)$$

这一事实直接源于用重要性重新加权项重写这个替代目标：

$$\nabla_\theta J_\theta^{\text{token}} = \nabla_\theta E_x \left[\sum_{t=1}^L E_{y \sim \tilde{\pi}_t(y|x)} [r(y|x)] \right] \quad (52)$$

$$= \nabla_\theta E_x \left[\sum_{t=1}^L E_{y \sim \pi_0(y|x)} \left[\frac{\pi_\theta(y_t | x, y_{<t})}{\pi_0(y_t | x, y_{<t})} r(y|x) \right] \right] \quad (53)$$

$$= E_x \left[E_{y \sim \pi_0(y|x)} \left[\sum_{t=1}^L \frac{\pi_\theta(y_t | x, y_{<t})}{\pi_0(y_t | x, y_{<t})} r(y|x) \nabla_\theta \log \pi_\theta(y_t | x, y_{<t}) \right] \right], \quad (54)$$

其中最后一步来自我们在整个作业中一直使用的对数导数技巧。

这个替代目标让我们直观地了解代币级别重新加权引入的偏差。首先，对于第 t 项，前缀 $y_{<t}$ 是从 π_0 而不是 π_θ 中提取的，因此它可能不能代表当前模型中的前缀。其次，在我们从当前策略 π_θ 中采样令牌 y_t 后，后缀 $y_{>t}$ 是从 π_0 而不是从 π_θ 中提取的。换句话说，当目标评估操作 y_t 如何影响未来步骤时，它是根据过时策略 π_0 的行为而不是当前策略 π_θ 进行此评估。这两种偏差都可以通过序列水平来纠正

重新加权，正确地重新加权前缀和后缀（以增加方差为代价）。直观上，随着 π_θ 距离 π_0 越来越远，这些偏差会更加严重。

问题 (derive_surrogate_objectives): 导出重要性重新加权方法的替代目标 (2 分)

在上面，我们看到代币级别的重要性重新加权优化了替代策略下的预期奖励，其中我们从旧策略中对除一个位置之外的所有位置进行采样。假设我们考虑以下策略梯度估计器给出的“成对”重要性重新加权方法：

$$\sum_{t=1}^{\frac{L}{2}} \frac{\pi_\theta(y_{2t-1} | x, y_{<2t-1}) \pi_\theta(y_{2t} | x, y_{<2t})}{\pi_0(y_{2t-1} | x, y_{<2t-1}) \pi_0(y_{2t} | x, y_{<2t})} r(y | x) \nabla_\theta [\log(\pi_\theta(y_{2t-1} | x, y_{<2t-1}) \pi_\theta(y_{2t} | x, y_{<2t}))] \quad (55)$$

其中 π_θ 是我们当前的策略， π_0 是我们过时的采样策略， $y \sim \pi_0(y | x)$ 。该估计器正在优化什么替代目标？

可交付成果：问题参数子集的表达式，并附有推导。

6.2.2 裁剪

除了代币级别的重新加权之外，PPO 和 GRPO 引入的另一个启发式是重要性权重剪裁。正如我们在上面看到的，随着 π_θ 远离 π_0 ，重要性重新加权的方差变得更加严重。我们的替代目标告诉我们，代币级别重新加权的偏差也变得更加严重。因此，为了保持离策略 RL 稳定，我们需要确保 π_θ 保持接近 π_0 。

确保 π_θ 保持接近 π_0 的最简单方法是减少每个推理批次的训练步骤数，但某些推理批次可能允许更多步骤。为了提高算法的性能，我们希望在推理批次允许的情况下采取尽可能多的步骤。为了实现这一目标，一系列方法涉及对具有大或小的重要性权重的术语进行剪裁。

最近的论文在如何实现裁剪的细节上存在分歧，但在本次作业中，我们将实现 PPO 的裁剪方法，该方法一直延续到 GRPO，并且似乎是主要方法。首先，将代币级别重新加权与标准 GRPO 目标相结合，产生以下损失函数：

$$J_\theta^{\text{GRPO-off-policy-noclip}} = \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{1}{\log(y^{(i,j)})} \frac{r(y^{(i,j)} | x^{(i)}) - \mu_i}{\text{std}_i} \sum_{t=1}^{\log(y^{(i,j)})} \frac{\pi_\theta(y_t | x, y_{<t})}{\pi_0(y_t | x, y_{<t})}, \quad (56)$$

其中样本是从我们过时的推理策略 π_0 中抽取的。您可以检查微分此目标为我们提供了上面导出的令牌重新加权策略梯度估计器，但具有 GRPO 优势和序列标准化。

我们准备为此目标添加裁剪。令 $A^{(i,j)} = \frac{r(y^{(i,j)} | x^{(i)}) - \mu_i}{\text{std}_i}$ 表示提示 i 上响应 j 的优势，并令 $w_t^{(i,j)} = \frac{\pi_\theta(y_t | x, y_{<t})}{\pi_0(y_t | x, y_{<t})}$ 表示第 t 个重要性重新加权项，该目标的剪辑版本如下：

$$J_\theta^{\text{GRPO-off-policy-clip}} = \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{1}{\log(y^{(i,j)})} \sum_{t=1}^{\log(y^{(i,j)})} \min(A^{(i,j)} w_t^{(i,j)}, A^{(i,j)} \text{clip}(w_t^{(i,j)}, [1 - \epsilon, 1 + \epsilon])), \quad (57)$$

其中 $\text{clip}(w, [1 - \varepsilon, 1 + \varepsilon]) = \min(\max(w, 1 - \varepsilon), 1 + \varepsilon)$ 将重要性权重修剪到区间 $[1 - \varepsilon, 1 + \varepsilon]$ 。编写此目标的一种稍微更直观的方法如下 [J.阿希姆, 2018]:

$$J_{\theta}^{\text{GRPO-off-policy-clip}} = \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{1}{\text{len}(y^{(i,j)})} \sum_{t=1}^{\text{len}(y^{(i,j)})} \begin{cases} \min(w_t^{(i,j)}, 1 + \varepsilon) A^{(i,j)} & \text{if } A^{(i,j)} \geq 0 \\ \max(w_t^{(i,j)}, 1 - \varepsilon) A^{(i,j)} & \text{if } A^{(i,j)} < 0 \end{cases} \quad (58)$$

在这个目标下，模型被激励去增加具有高优势的动作的权重，直到它对该动作 $1 + \varepsilon$ 的权重比旧策略更大。此时， $\min$ 采用 $1 + \varepsilon$ 路径，并且通过它进行微分会产生零梯度。负优势动作是类似的：模型被激励去降低动作的权重，直到它对该动作 $1 - \varepsilon$ 的权重小于旧策略，此时该动作的梯度变为零。因此，采用梯度，我们得到以下策略梯度估计器：

$$\nabla_{\theta} J_{\theta}^{\text{GRPO-off-policy-clip}} = \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{1}{\text{len}(y^{(i,j)})} \sum_{t=1}^{\text{len}(y^{(i,j)})} \text{mask}_t^{(i,j)} w_t^{(i,j)} A^{(i,j)} \nabla_{\theta} \log \pi_{\theta}(y_t | x, y_{<t}) \quad (59)$$

具有遮罩功能

$$\text{mask}_t^{(i,j)} = \begin{cases} \mathbb{1}\{w_t^{(i,j)} < 1 + \varepsilon\} & \text{if } A^{(i,j)} \geq 0 \\ \mathbb{1}\{w_t^{(i,j)} > 1 - \varepsilon\} & \text{if } A^{(i,j)} < 0 \end{cases} \quad (60)$$

它掩盖了积极优势的大重要性权重，并掩盖了消极优势的小重要性权重。

现在我们有在损失函数中实现 `importance_reweighting_method = "noclip"` 和 `importance_reweighting_method = "grpo"` 所需的数学，它们应该对应于 $J_{\theta}^{\text{GRPO-off-policy-noclip}}$ 和 $J_{\theta}^{\text{GRPO-off-policy-clip}}$ 。与在策略情况一样，从损失实现中返回负目标，以便梯度下降执行预期的梯度上升更新。

问题 (`compute_policy_gradient_loss_off_policy`): 具有令牌级别重新加权的离策略策略梯度 (1 分)

可交付成果：更新您的方法 `compute_policy_gradient_loss` 以支持 `importance_reweighting_method = "noclip"` 或 `"grpo"`，它们使用 `old_log_probs` 和 `cliprange` 参数。`old_log_probs` 表示用于生成推出的模型下的每个令牌的日志概率，您必须在训练脚本中添加几行来计算这些概率。`cliprange` 表示削波强度参数 ε 。

要测试您的代码，请实施 [`adapters.run_compute_policy_gradient_loss`]。然后运行 `uv run pytest -k test_compute_policy_gradient_loss_off_policy` 并确保测试通过。

注意：虽然 PPO 中的剪裁是为了防止当前策略偏离旧策略太远，但它也可以被视为减少重要性重新加权计量方差的一种方法，因为我们在压缩太大的重要性权重项。如果我们不太关心偏离旧策略，并且希望在更新中更加积极，那么控制方差的一种更简单、更直接的方法是直接对梯度估计器中的重要性权重进行上限剪辑：

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{1}{\text{len}(y^{(i,j)})} \sum_{t=1}^{\text{len}(y^{(i,j)})} \min(w_t^{(i,j)}, 1 + \varepsilon) A^{(i,j)} \nabla_{\theta} \log \pi_{\theta}(y_t | x, y_{<t}), \quad (61)$$

与 PPO/GRPO 不同的是，我们仍然通过增加超过 $1 + \varepsilon$ 的动作来采用非零梯度。这种更简单的裁剪过程是在 CISPO [MiniMax et al., 2025] 中提出的（注意：它们还通过每组标记计数而不是每序列长度进行标准化，但为了简单起见，我们编写了序列标准化目标）。或者，欢迎您尝试 CISPO 并将其与其他剪切方法进行比较。

6.3 GSPO

正如我们在上面看到的，令牌级别的重新加权是有偏差的，因为它忽略了前缀和后缀的重新加权。在 GSP O 论文中 [C. Cheng et al., 2025]，作者发现 GRPO 不稳定，并认为我们应该进行序列级重新加权。为了解决序列级重要性权重是 L 项的乘积这一事实，他们只需将表达式取 $\frac{1}{L}$ 次方即可。换句话说，它们通过序列的几何平均重要性权重而不是乘积来重新加权。结果是以下损失函数：

$$J_{\theta}^{\text{GRPO-off-policy-gspo}} = \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \min(A^{(i,j)} s^{(i,j)}, A^{(i,j)} \text{clip}(s^{(i,j)}, [1 - \varepsilon, 1 + \varepsilon])), \quad (62)$$

现在有一个跨时间步共享的序列级重要性权重 $s^{(i,j)}$ ：

$$s^{(i,j)} = \left(\prod_{t=1}^{\text{len}(y^{(i,j)})} \frac{\pi_{\theta}(y_t | x, y_{<t})}{\pi_0(y_t | x, y_{<t})} \right)^{\frac{1}{\text{len}(y^{(i,j)})}}. \quad (63)$$

请注意，如果没有 $\frac{1}{\text{len}(y^{(i,j)})}$ 的能力并且没有裁剪，这个目标就变成了序列级重要性重新加权策略梯度损失。采用几何平均值和剪裁可以减少方差，但会产生偏差。另一个注意事项是，在采用梯度时，由于几何平均功率，我们自然会得到序列长度归一化（为简洁起见，在下面的等式中忽略了裁剪）：

$$\nabla_{\theta} J_{\theta}^{\text{GRPO-off-policy-gspo}} = \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G A^{(i,j)} \nabla_{\theta} s^{(i,j)} \quad (64)$$

$$= \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G A^{(i,j)} s^{(i,j)} \frac{1}{\text{len}(y^{(i,j)})} \sum_{t=1}^{\text{len}(y^{(i,j)})} \nabla_{\theta} \log \pi_{\theta}(y_t | x, y_{<t}). \quad (65)$$

因此，如果您想将 GSPO 应用于您最喜欢的常数归一化 RL 算法（可选），您将需要更改序列级重要性权重中的指数。

问题 (**think_about_importance_reweighting**): 考虑重要性重新加权 (2 分)

考虑离策略 RL 的三种可能的重要性重新加权策略：(a) 无重要性重新加权，(b) PPO/GRPO 风格的截断标记级别重要性重新加权，以及 (c) GSPO 风格使用几何平均值的截断序列级别重要性重新加权。就权衡偏差和方差而言，每种方法处于哪个范围？您能想到一种方法在直观上可能比其他方法更好的情况吗？

可交付成果：几句话评论。

在下面的问题中，我们将实现 GSPO 损失。请记住以数值稳定的方式计算几何平均值。

问题 (`compute_policy_gradient_loss_off_policy_gspo`)： 具有序列级重新加权的离策略策略梯度 (1 分)

可交付成果：更新您的方法 `compute_policy_gradient_loss` 以支持 `importance_reweighting_method = "gspo"`，它使用 `old_log_probs` 和 `cliprange` 参数。`old_log_probs` 表示用于生成推出的模型下的每个令牌的日志概率，您必须在训练脚本中添加几行来计算这些概率。`cliprange` 表示削波强度参数 ϵ 。如上所述，返回负目标，以便最小化损失执行梯度上升。

要测试您的代码，请实施 `[adapters.run_compute_policy_gradient_loss]`。然后运行 `uv run pytest -k test_compute_policy_gradient_loss_off_policy_gspo` 并确保测试通过。

6.4 实验

我们现在拥有更新 `grpo_train_step` 方法以支持离策略训练所需的组件。

问题 (`grpo_train_step_off_policy`)： 离策略 GRPO 训练步骤 (2.5 分)

可交付成果：更新您的 `grpo_train_step` 方法以支持离策略参数 (`importance_reweighting_method`、`old_log_probs`、`cliprange`)。该函数现在应该支持全部参数范围。

要测试您的代码，请实施 `[adapters.run_grpo_train_step]`。然后运行 `uv run pytest -k test_grpo_train_step_off_policy` 并确认通过。

使训练脱离策略实际上会使其更快吗？我们会在多大程度上失去稳定性？哪种重新加权/裁剪方法最好？我们将在下面的实验中探讨这些问题。请保持与之前的 RL 运行相关的超参数固定，但运行现在应该是 32 倍的关闭策略（因此我们为每个推理批次采取 32 个训练步骤）：

```
rollout_batch_size = 256
train_batch_size = 8
gradient_accumulation_steps = 1
```

此外，作为默认值，您可以使用以下取自原始论文的剪切范围值（即上面推导中的 ϵ ）。或者，也欢迎您自己调整这些值。

- `offpolicy_clip`: `cliprange = 0.2`
- `offpolicy_gspo`: `cliprange = 3e-4`

请注意，您还必须在训练脚本中添加几行来计算 `old_log_probs`，以传递到重新加权的损失中。

问题 (**grpo_experiments_off_policy**): 比较不同离策略算法的性能 (8 B200 小时) (10 分)

相对于标准 GRPO 训练运行 (根据您选择的学习率) 保持超参数固定, 运行以下变体, 每个变体使用 4 随机种子。您应该使用零样本 `r1_zero` 提示符。如果您愿意, 可以从上一节中选择您最喜欢的 RL 算法变体, 而不是坚持使用标准版本。

- `offpolicy_naive`: 不要使推理批大小和训练批大小相等 (均为 256), 而是将训练批大小减少到推理批大小的 $1/32$ ($\frac{256}{32} = 8$)。请记住以相同的因子减少梯度累积步骤的数量, 以便我们继续充分利用我们的 GPU。保留 `importance_reweighting_method = "none"`。
- `offpolicy_noclip`: `offpolicy_naive` 与 `importance_reweighting_method = "noclip"`。
- `offpolicy_clip`: `offpolicy_naive` 与 `importance_reweighting_method = "grpo"`。
- `offpolicy_gspo`: `offpolicy_naive` 与 `importance_reweighting_method = "gspo"`。

除了您已经记录的指标之外, 请确保还记录剪辑分数。

与您之前运行的完全同策略 GRPO 运行相比, 这些方法的执行情况如何? 偏离策略如何影响训练稳定性和运行之间的差异? 这两种裁剪方法在裁剪分数方面有何不同, 哪一种看起来更稳定? 您认为是否有一些方法可以从更多的超参数调整中受益? 根据运行之间的差异, 您对自己的发现有多大信心?

可交付成果: 评论, 由参考指标的图表支持。

7 尝试你自己的策略梯度估计器

问题 (**try_your_own**): 尝试您自己的策略梯度估计器 (10 分)

现在您已经看到了文献中提出的各种策略梯度估计器, 以及它们背后的一些理论, 是时候提出您自己的了! 以下是一些建议:

- 您可以尝试不同的优势估计器, 对提示进行不同的重新加权。
- 您可以尝试不同的重要性重新加权策略。
- 您可以在优势估计器中尝试不同的奖励基线。

提出您自己的策略梯度估计器。然后, 在 GSM8K 上的 OLMo-2-0425-1B 的相同 RL 设置上运行它, 并将其与我们已经尝试过的基线方法进行比较, 使用多个随机种子并保持问题参数固定以使结果具有可比性。请仅更改与迄今为止尝试的方法相关的一件事 (即, 此作业中应该至少涵盖一种先前的方法, 以便您的方法仅更改一件事)。

您提议的变革背后的理论依据或直觉是什么? 您提出的更改是否超出了基线?

可交付成果: 评论, 由参考指标的图表支持, 以及可选的数学推导 (如果它们有助于证明您的方法)。

参考书目

- [1] J. Li *et al.*, “DataComp-LM: 寻找下一代语言模型训练集。” [在线的]。可用: <https://arxiv.org/abs/2406.11794> [2] T. OLMo *et al.*, “2 OLMo 2 Furious”。 [在线的]。可用: <https://arxiv.org/abs/2501.00656> [3] K. Cobbe *et al.*, “训练验证者解决数学应用题。” [在线的]。可用: <https://arxiv.org/abs/2110.14168> [4] DeepSeek-AI *et al.*, “DeepSeek-R1: 通过强化学习激励法学硕士的推理能力。” [在线的]。可用: <https://arxiv.org/abs/2501.12948> [5] W. Kwon *et al.*, “使用 PagedAttention 进行大型语言模型服务的高效内存管理”。 2023. [6] Z. Liu *et al.*, “理解 R1-类零训练: 一个批判性的视角。” [在线的]。可用: <https://arxiv.org/abs/2503.20783> [7] OpenAI *et al.*, “OpenAI o1 系统卡”。 [在线的]。可用: <https://arxiv.org/abs/2412.16720> [8] Z. Shao *et al.*, “DeepSeekMath: 突破开放语言模型中数学推理的极限。” [在线的]。可用: <https://arxiv.org/abs/2402.03300> [9] J. Achiam, “深度强化学习的旋转”, 2018. [10] N. Lambert, “来自人类反馈的强化学习”。 [在线的]。可用: <https://rlhfbook.com/> [11] R. J. Williams, “用于联结强化学习的简单统计梯度跟踪算法”, *Machine Learning*, 卷。 8, 不。 3-4, 第 229-256 页, 1992 年, doi: 10.1007/BF00992696. [12] F. Tajwar *et al.*, “最大似然强化学习”。 [在线的]。可用: <https://arxiv.org/abs/2602.02710> [13] J. Schulman, F. Wolski, P. Dhariwal, A. Radford 和 O. Klimov, “近端策略优化算法”。 [在线的]。可用: <https://arxiv.org/abs/1707.06347> [14] T. Degris, M. White 和 R. S. Sutton, “偏离政策的演员评论家”。 [在线的]。可用: <https://arxiv.org/abs/1205.4839> [15] J. Achiam, “简化的 PPO-Clip 物镜”。 [在线的]。可用: <https://drive.google.com/file/d/1PDzn9RPvaXjJFZkGeapMHbHGiWWW20Ey/view> [16] MiniMax *et al.*, “MiniMax-M1: 通过闪电注意力有效扩展测试时计算。” [在线的]。可用: <https://arxiv.org/abs/2506.13585> [17] C. Cheng *et al.*, “组序列策略优化”。 [在线的]。可用: <https://arxiv.org/abs/2507.18071>